COMP10002 FOUNDATIONS OF ALGORITHMS

Assignment 1: Attention Is All You Need

Compiled printable version of the web-first specification

COURSE

COMP10002 Foundations of Algorithms

TERM

Semester 1, 2026

SOURCE

COMP10002 A1 Web Spec

VERSION

1.0

This should only be used as a backup, as it is a static resource. Always verify the version number on the specification matches the newest on the webpage if the website is accessible. This contains the main spec, with useful concept explainers, support appendices and optional background collected separately afterwards.

CONTENTS

Table of contents

Main specification

1. Assignment Specification

Extra Explanations

1. Tokens and Embeddings
2. One-Hot Encoding
3. Q, K, V Projections
4. Dot Products and Scores
5. Attention
6. Causal and Padding Masks
7. Softmax and Stable Softmax
8. KV Cache

Resources

1. Appendix A. Annotated Scaffold
2. Appendix B. What an Input File Looks Like
3. Appendix C. Strings in C

4. Appendix D. Checking and Debugging Your C Code
5. Appendix E. Toy Demo

Optional background appendices

1. Appendix G. Reading Transformer Paper Notation
2. Appendix H. Transformer Explainer
3. Appendix I. Reading LLM Architecture Diagrams
4. Appendix J. Transformer History
5. Appendix K. Parameters and Model Scale
6. Appendix L. How This Assignment Was Built

SECTION

COMP10002 FOUNDATIONS OF ALGORITHMS

Assignment 1: Attention Is All You Need

A web-first version of the assignment spec. You should be able to read this page straight through; the linked explainers are there only when you want more intuition. If there is ever an inconsistency between this main page and a linked explainer, this page is most likely the correct information, so treat the other pages as supplemental unless otherwise advised.

Semester 1, 2026 Due: Monday 27th April, 6PM AEST Version: 1.0

> **Important notes**
>
> - Submission is a single file: `a1.c`, through gradescope.
> - Start from the provided scaffold and fill in the `TODO` functions.
> - Do not change the stage header lines or print helpers.
> - Do not add extra `printf` debugging output.
> - Your code must compile on the marking system or the testcase component cannot run.

> **⚠ Compilation errors matter**
>
> Because the testcase mark is autograded, your program must compile on the marking system. If it does not compile, it cannot be run and **you will receive zero for correctness**.

> **How to use this page**
>
> You can read this page from top to bottom like the assignment handout. If a term such as softmax, projection, or KV cache slows you down, use the inline link for a deeper explanation, then come back here.

> **Downloadable PDF**
>
> Prefer a printable version? Download the compiled project spec PDF. It bundles this main spec, the concept explainers, and the optional support pages as appendices.

What is this assignment about?

This assignment is inspired by how modern Large Language Models (LLMs) such as chatbots work.

An LLM is trained to predict the **next token** in a sequence. Conceptually, given a prompt

$$\texttt{token}_0, \texttt{token}_1, \ldots, \texttt{token}_{n-1},$$

the model produces a probability distribution for the next token $\texttt{token}_n$.

Think of it like autocomplete: based on what you typed so far, what is most likely to come next?

What a real LLM would do, at a high level

You do not need the exact ML details, but a real LLM roughly:

1. tokenises text into token IDs
2. looks up token embeddings
3. runs many Transformer layers
4. uses attention inside those layers
5. produces scores over a vocabulary
6. chooses the next token, appends it, and repeats

What this assignment does instead

You are **not** implementing a full chatbot. You are implementing a core part used inside them: a simplified single-head self-attention block plus a final KV-cache generation step.

If you want the plain-English picture of what attention is doing before you get into the loops, read the Attention explainer.

To keep the programming focused and testable:

- you begin with a small Stage 1 representation exercise that turns a token list into one-hot vectors
- the later attention stages use the provided prompt and generated embeddings directly as numeric rows
- you are given the parameter matrices `Wq`, `Wk`, and `Wv` directly as numbers
- you compute attention scores, weights, and outputs exactly
- you apply masking rules and stable softmax
- you do not predict English words

If the maths still feels abstract, that is fine. The implementation requirements are stated explicitly below, and the linked explainer pages fill in the intuition when you need it.

Where the numbers in this assignment come from

A real language model starts with text, not matrices. Very roughly, the pipeline is:

1. split the text into tokens
2. turn each token into an embedding vector
3. run attention over those vectors to build context-aware outputs

4. use the final vector to help predict what token should come next

This assignment still simplifies a lot, but it lets you practise the representation idea separately from the later attention maths. You do not implement a full tokenizer, embedding-table lookup, vocabulary-scoring layer, or training process.

The project therefore has two related but separate “before attention” pieces:

- a short token list used in Stage 1, where you build and print a one-hot embedding table
- the prompt and generated embedding rows used by the later attention stages

That means Stage 1 is a warm-up representation exercise on tokens, while Stages 2 to 6 operate on the provided numeric embedding matrices.

That means:

- each row of the prompt matrix is one token embedding
- `n` is the number of prompt tokens
- `d` is the number of components in each embedding vector
- the prompt positions still have an order, so “earlier token” and “later token” matter

The core job of attention is then: for one position, look back at the earlier positions that are allowed, decide which ones matter most, and blend their information into a new output row.

> **Recommended visual introduction**
>
> If you want one strong visual introduction before or during the assignment, watch 3Blue1Brown: Attention in transformers.
>
> It is especially useful for building intuition about what single-head self-attention is doing before you get lost in the matrix arithmetic. Just note that they show embedding vectors as columns instead of rows like this assignment!

The whole job in one pass

Your program follows one continuous pipeline:

1. read the token list, mask, embedding rows, and parameter matrices from the input file
2. build and print the Stage 1 one-hot embeddings from the token list
3. project the provided prompt embeddings into query, key, and value matrices
4. compare prompt queries against prompt keys to get attention scores
5. turn those scores into attention weights with masking and stable softmax

6. use the weights to blend the value vectors into prompt outputs
7. extend the same logic to generated tokens while reusing cached keys and values

If you keep that pipeline in mind, the stage descriptions later on should read as one algorithm being built step by step, not as unrelated tasks.

Mental Model For The Stages

Keep this mental model in mind while reading the stages:

- a prompt position starts as “just this token’s embedding”
- by the end of Stage 5, that row has become “this token, rewritten using the earlier rows that mattered to it”
- Stage 6 applies the same attention idea during generation, but reuses cached keys and values instead of rebuilding old work

If you lose the thread while coding, come back to these two questions:

- which earlier rows is this position allowed to use?
- how much should each allowed row contribute to the new output row?

In a full model there would be more machinery around this block, such as tokenisation , embedding lookup , many repeated layers, and a final vocabulary-scoring step. Here you only implement the attention part in the middle.

Optional background and visual explainers

If you want more background, worked examples, or visual explanations, these pages are useful:

- Concept explainers for attention, projections, masking, softmax, and related ideas
- Transformer explainer for a one-page overview of the larger model architecture
- Concepts for both the required explainers and the optional demos, tooling, and background pages
- The Illustrated Transformer for a visual walkthrough of the model structure
- Attention Is All You Need for the original Transformer paper

Those pages support this spec. They are not required pre-reading.

Glossary and notation

This section collects the vocabulary and notation that help you read the assignment, before you get into the stage-by-stage implementation details.

Glossary

TERM	MEANING
Token	A piece of text, sometimes a whole word and sometimes part of a word.
Prompt	The starting tokens you already have before generating new tokens.
Generate	Produce new tokens one at a time after the prompt.
Vector	A fixed-length list of numbers. In C, a 1D array of `double`.
Matrix	A table of numbers. In C, a 2D array of `double`.
Embedding	A vector representing a token as numbers.
Dot product	Multiply matching positions and add them up.
Score	A number saying how relevant one token is to another.
Softmax	A function that turns scores into weights that sum to 1.
Mask	A rule that says “ignore this position”.
Causal mask	The mask that stops row `i` from using any future position `j > i`.
Padding mask	The mask that ignores prompt positions marked as padding, usually where `mask[j] == 0`.
KV cache	Stored past keys and values used during generation.

Notation and conventions

Throughout this spec:

- We number tokens starting at 0 when describing arrays, so prompt indices are $0, 1, \ldots, n-1$.

- A vector of length **d** has components $0, \ldots, d-1$. For clarity we will show vectors as bold with an arrow above and use round brackets.
  For example $\vec{\mathbf{v}} = (1 \quad 2)$, would be `double v[] = {1 , 2}` in C.

- For clarity we will show matrices as bold and write it using square brackets.
  For example $\mathbf{M} = [1 \quad 2]$ would be `double M[][] = { {1 , 2} }` in C. If we write $\vec{\mathbf{M}}_i$ we mean the vector corresponding to row i in $\mathbf{M}$, in C that is `M[i]`

- "$\sum$" means "add up a bunch of terms" (like a loop). For example, $\sum_{t=0}^{d-1} a[t] \cdot b[t] = a[0]b[0] + \cdots + a[d-1]b[d-1]$ means "for each **t**, multiply, then add the results".

- Subscripts like $Q_{i,t}$ for a matrix $\mathbf{Q}$ mean "row i, component t". In C, that is `Q[i][t]`.
- `Masked` means "treated as if it does not exist": its weight must be `0.0`, and in Bonus Stage 5 you must not compute a dot product for it.

Dot products, vectors and matrices

If we have vectors $\vec{\mathbf{a}} = (1 \quad 2)$ and $\vec{\mathbf{b}} = (3 \quad 4)$, then the dot product is given by

$$\vec{\mathbf{a}} \bullet \vec{\mathbf{b}} = (1 \quad 2) \bullet (3 \quad 4) = 1 \cdot 3 + 2 \cdot 4 = 11.$$

Because we will be working with matrices, it is convenient if we define our dot product as a matrix multiplication between 2 matrices. For a matrix multiplication to be defined the number of columns in the first matrix, and the number of rows in the second matrix must be equal. The dimension of the resulting matrix, will be the number of rows of the first matrix and the number columns of the second matrix. In the case below it will be 1 by 1 (just a number, exactly as our dot product between vectors requires).

$$\vec{\mathbf{a}} \bullet \vec{\mathbf{b}} = \mathbf{a}\mathbf{b}^T = [1 \quad 2] \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

By the standard rules of matrix multiplication, this gives us the same result as our dot product. So this is equivalent to the dot product between the vector corresponding to the row of the first matrix, and the column of the second matrix.

In C our matrices **a** and **b** would be `double a_matrix[][] = { {1, 2} }` and `double b_matrix[] = { {3, 4} }`. Then $\mathbf{b}^T$ would be the 2D matrix `double bT_matrix[][] = { {3}, {4} }`. Here $\mathbf{b}^T$ means the transpose of **b**, where transpose just means "flip it across the diagonal so rows become columns and vice versa", which is required to ensure the matrix multiplication is defined.

Note that we can't directly extract a 'column vector' from the $\mathbf{b}^T$ matrix, like we can with a row (eg with `a_matrix[0]`), so it can be convenient to do operations in terms of matrices.

In this assignment, dot products appear inside matrix-style calculations, so it is useful to remember that each output component in a projection is "one row dotted with one column".

If you want help reading the compact notation used in the Transformer paper and later papers, read the paper-notation explainer. If you want a visual explanation of row-vector times matrix multiplication, read the projection explainer.

Getting started

This section is the fast setup pass: what you have, what the scaffold already does, and what constraints matter before you start coding.

> **You have been given**
>
> - a scaffold `a1.c` with a working `main` and `TODO` functions
> - sample inputs in `test_data/`
> - matching expected outputs in `test_data/`

> **The scaffold already**
>
> - prints the required stage headers and row formatting
> - calls your functions in the correct order
> - allocates arrays at safe maximum sizes

> **You should**
>
> - fill in only the functions marked `/* TODO: ... */`
> - keep all arrays within the provided maximum sizes
> - use `double` for calculations
> - include `<math.h>` and compile with `-lm`

> **You must not**
>
> - change the stage header lines

> - add extra output
> - change `print_vector`

> **Inspect the scaffold first**
>
> Before you start coding, spend some time familiarising yourself with `a1.c`:
>
> - what arrays already exist
> - what each function is meant to compute
> - what gets printed

> **How to get going**
>
> Start by reading `a1.c`, then implement the stages in order.
>
> Begin with read_input(...) , because every later stage depends on having the input stored correctly. Then get the Stage 1 embedding output working, then compute Stage 2 projections, Stage 3 scores, Stage 4 weights, Stage 5 outputs, and finally Stage 6 generation with the KV cache.
>
> After each stage, compile and run `test0` before moving on.

If you need more intuition while coding, use the linked explainers at the point where they become relevant: one-hot encoding for Stage 1, tokens and embeddings for the input story, projections for Stage 2, dot products and masking for Stage 3, softmax for Stage 4, attention output for Stage 5, and KV cache for Stage 6.

If reading `a1.c` itself is slowing you down, use the Annotated scaffold. The function pulls in the stage sections below jump straight to the matching scaffold section there.

Stage descriptions

The most useful way to read this section is as one algorithm unfolding in order. Stage 0 loads the token list and numeric snapshot. Stage 1 builds and prints one-hot embeddings. Stage 2 turns the provided prompt embeddings into `Q`, `K`, and `V`. Stages 3 to 5 perform attention over the prompt. Stage 6 then applies the same ideas during generation using the cache.

Each stage names the scaffold function you should implement first when there is one. Stage 1 is the small embedding-construction stage that now sits in front of the attention pipeline.

Stage 0: Reading the input

If you want to see what a real input file looks like before reading the format rules, open What an input file looks like.

Implement this scaffold function first: read_input(...) . Every later stage depends on storing the input snapshot correctly.

Read from `stdin` in this order:

1. `n d g text_len` . The three attention sizes, followed by the number of tokens in the Stage 1 text.
2. `text_len` whitespace-separated tokens. This is the Stage 1 text that you turn into the one-hot embedding table.
3. `n` integers for `mask` . These say which prompt positions are real and which ones are padding.
4. `n` prompt embeddings of length `d` . These rows are the numeric prompt matrix used by the later attention stages.
5. `g` generated embeddings of length `d` . These rows are used later in the generation stage.
6. `d` lines for `Wq` the query projection matrix.
7. `d` lines for `Wk` the key projection matrix.
8. `d` lines for `Wv` the value projection matrix.

Bounds

SYMBOL	MEANING	RANGE
`n`	number of prompt tokens	`1 <= n <= 64`
`d`	embedding dimension	`1 <= d <= 64`
`g`	number of generated tokens	`0 <= g <= 32`
`text_len`	number of tokens in the Stage 1 text	`1 <= text_len <= 64`

Use `scanf`. Do not open files in your C program.

Whitespace-separated input means spaces and newlines are equivalent for `scanf`.

Stage 1: Creating embeddings

This stage is the small representation exercise before the attention maths begins. You take the input text, treat each whitespace-separated piece as a token, build the unique-token list, sort it lexicographically, and then print one one-hot vector per token in that sorted list.

The scaffold function for this stage is create_embeddings(...) . If you use `qsort(...)`, the comparison helper sits right beside it in the scaffold.

The core job in this stage is:

1. sort the token array stored by Stage 0 lexicographically
2. remove duplicate tokens from the sorted array
3. give each token a vector that is all zero except for one `1` based on the index in the sorted array
4. print the token together with its one-hot embedding as per the Output Format

That is the one-hot encoding idea. It is a simple bridge from categories to numbers, and the one-hot explainer shows it visually.

If this stage makes sense in principle but you have not really worked with C strings yet, use Strings in C. It is meant as a short crash course on the string operations this stage relies on.

This stage is the simple practice version of token representation.

> **Important Stage 1 Difference**
>
> The Stage 1 vector length is the number of unique tokens in the sorted list, **not** the prompt embedding dimension `d` used in the later attention stages.

You can use either `qsort(...)` or your own sorting code for the lexicographic ordering step.

How this stage relates to the later attention stages

The later attention stages do **not** depend on these Stage 1 vectors as input. This stage is there to make the idea of "text becomes vectors" concrete before the rest of the assignment switches to the provided numeric embedding matrices.

The important representation rule is:

- the sorted unique-token array is the embedding table
- the token's position in that sorted array decides where the `1` goes
- every other position in that embedding is `0`

That means the embedding dimension for this stage is "number of unique tokens in the sorted list".

> ▼ **WORKED EXAMPLE: A TOY EMBEDDING TABLE**
>
> If the sorted token list is
>
> ```
> algorithms, best, foundations, of, subject, the, to, welcome
> ```
>
> the one-hot vectors are:

```
algorithms   →  ( 1  0  0  0  0  0  0  0 )
best         →  ( 0  1  0  0  0  0  0  0 )
...          →  ( .  .  .  .  .  .  .  . )
welcome      →  ( 0  0  0  0  0  0  0  1 )
```

This is not how modern trained models usually build embeddings, but it is a clean way to show how text can become vectors before attention begins.

For a smaller three-token case, the printed output would look like:

```
Stage 1: Create Embeddings
"algorithms" -> (1 0 0)
"subject" -> (0 1 0)
"the" -> (0 0 1)
```

Stage 2: Projections

In an LLM once the prompt text has been tokenised, it will then be represented as a sequence of embedding vectors.

In the stage, we provide you a sequence of embedding vectors corresponding to a tokenised prompt, and not rely on the previous stage to keep it seperate. If we did use the previous stage, for a given prompt, you would look up the embedding for each token, and pass an array of those embeddings to this stage. Notably the embedding for every token is the same regardless of where in the prompt it is, despite the location of words, in relation to others, altering the meaning. This is where attention comes in!

Once the input is stored, our first computation to implement attention is to build the prompt query, key, and value matrices .

Before reading on make sure you are familar with the notation

The input prompt embeddings are the rows of a matrix $\mathbf{X}$. To get queries, keys, and values, you project the same prompt matrix three times with three different matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_{\mathbf{q}} \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_{\mathbf{k}} \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_{\mathbf{v}}$$

What Q, K, and V are, and why are there three of them.

How do these matrices link to the C Scaffold?

- $\mathbf{X}$ is the prompt embedding matrix: `prompt`. Each row of the matrix corresponds to one prompt token embedding vector.
- $\mathbf{W_q}, \mathbf{W_k}, \mathbf{W_v}$ are the weight matrices. In C these are `Wq`, `Wk`, `Wv` stored as 2D arrays.
- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are the prompt projection vector matrices. In C these are `q_matrix`, `k_matrix`, `v_matrix` stored as 2D arrays.

Implement the scaffold function for this stage: compute_projection(...) .

If $\mathbf{X}$ is the input matrix, $\mathbf{W}$ is one of the projection matrices, and $\mathbf{Y}$ is the projected output matrix, then:

$$Y_{i,j} = \sum_{u=0}^{d-1} X_{i,u} W_{u,j} \qquad \mathbf{Y} = \begin{bmatrix} Y_{0,0} & \dots & Y_{0,d-1} \\ \vdots & \ddots & \vdots \\ Y_{n-1,0} & \dots & Y_{n-1,d-1} \end{bmatrix}$$

Be careful to reset the sum to `0` for each output component.

This function is called three times to compute prompt $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$

▼ **WORKED EXAMPLE: ONE TOKEN EMBEDDING VECTOR**

For a single token vector $\vec{\mathbf{x}}$ of length `d`, the projected vector $\vec{\mathbf{y}}$ is defined component by component by:

$$\vec{\mathbf{x}} = (x_0 \quad \dots \quad x_{d-1}), \qquad y_j = \sum_{u=0}^{d-1} x_u W_{u,j}, \qquad \vec{\mathbf{y}} = (y_0 \quad \dots \quad y_{d-1}).$$

For an example with `d = 2`, we can equivalently write our component wise sum as a matrix multiplication. We will define the matrix $\mathbf{X} = [\vec{\mathbf{x}}]$ so we can write things in terms of matrix multiplication:

$$\mathbf{X} = [x_0 \quad x_1] \qquad \mathbf{W} = \begin{bmatrix} W_{0,0} & W_{0,1} \\ W_{1,0} & W_{1,1} \end{bmatrix}$$

Then

$$\begin{aligned} \mathbf{Y} &= [x_0 \quad x_1] \begin{bmatrix} W_{0,0} & W_{0,1} \\ W_{1,0} & W_{1,1} \end{bmatrix} = \left[[x_0 \quad x_1] \begin{bmatrix} W_{0,0} \\ W_{1,0} \end{bmatrix} \quad [x_0 \quad x_1] \begin{bmatrix} W_{0,1} \\ W_{1,1} \end{bmatrix} \right] \\ &= [(x_0 \quad x_1) \bullet (W_{0,0} \quad W_{1,0}) \quad (x_0 \quad x_1) \bullet (W_{0,1} \quad W_{1,1})] \\ &= [(x_0 \cdot W_{0,0} + x_1 \cdot W_{1,0}) \quad (x_0 \cdot W_{0,1} + x_1 \cdot W_{1,1})] \end{aligned}$$

This is the same loop pattern your code uses for every row of the prompt matrix.

▼ **WORKED EXAMPLE: TWO TOKEN EMBEDDING VECTORS**

In our previous example we treated one input vector as the row of a matrix, but we can extend this to have multiple input vectors, each as a different row, and our matrix multiplication is still defined, just the resulting matrix with have more rows in it!

> To get the query, key and value token matrices, we will do this projection operation three times with three different weight matrices:
>
> $$\mathbf{Q} = \mathbf{X}\mathbf{W_q}, \quad \mathbf{K} = \mathbf{X}\mathbf{W_k}, \quad \mathbf{V} = \mathbf{X}\mathbf{W_v}.$$
>
> We can now consider a whole matrix of `n` prompt embedding vectors (each is a row of $\mathbf{X}$) of dimension `d = 2` at once
>
> $$\begin{aligned}\mathbf{Y} = \mathbf{X}\mathbf{W} &= \begin{bmatrix} X_{0,0} & X_{0,1} \\ X_{1,0} & X_{1,1} \end{bmatrix} \begin{bmatrix} W_{0,0} & W_{0,1} \\ W_{1,0} & W_{1,1} \end{bmatrix} = \begin{bmatrix} [X_{0,0} \quad X_{0,1}] \begin{bmatrix} W_{0,0} \\ W_{1,0} \end{bmatrix} & [X_{0,0} \quad X_{0,1}] \begin{bmatrix} W_{0,1} \\ W_{1,1} \end{bmatrix} \\ [X_{1,0} \quad X_{1,1}] \begin{bmatrix} W_{0,0} \\ W_{1,0} \end{bmatrix} & [X_{1,0} \quad X_{1,1}] \begin{bmatrix} W_{0,1} \\ W_{1,1} \end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} (X_{0,0} \cdot W_{0,0} + X_{0,1} \cdot W_{1,0}) & (X_{0,0} \cdot W_{0,1} + X_{0,1} \cdot W_{1,1}) \\ (X_{1,0} \cdot W_{0,0} + X_{1,1} \cdot W_{1,0}) & (X_{1,0} \cdot W_{0,1} + X_{1,1} \cdot W_{1,1}) \end{bmatrix}\end{aligned}$$
>
> In this example we have `n = 2` token embedding vectors stored in our token embedding matrix: $(X_{0,0} \quad X_{0,1})$ and $(X_{1,0} \quad X_{1,1})$. We can see that in the output matrix, each row corresponds to the projected vector for the given token embedding vector in that row of the input. The computation in each row is independant of the other rows, so we can calculate the result for each row seperately, but it is notationally compact to store the vectors as rows of a matrix, and convenient when it comes to programming aswell!

Stage 3: Attention scores (prompt)

With $\mathbf{Q}$ and $\mathbf{K}$ available, the next step is to measure how strongly each key vector corresponding to prompt token j should attend to each query vector corresponding prompt token i. For this task you will be computing the **score** matrix, using the projections from the previous stage.

You should implement the scaffold function: compute_attention_scores_or_weights_prompt(...)

For each query vector corresponding to prompt token i ($\vec{\mathbf{Q}}_i$) and key vector corresponding to prompt token j ($\vec{\mathbf{K}}_j$), we compute an attention score. The score is a scaled dot product:

$$\text{score}_{i,j} = \frac{\vec{\mathbf{Q}}_i \bullet \vec{\mathbf{K}}_j}{\sqrt{d}} = \frac{\sum_{u=0}^{d-1} Q_{i,u} K_{j,u}}{\sqrt{d}} \qquad \mathbf{score} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} = \begin{bmatrix} \text{score}_{0,0} & \dots & \text{score}_{0,n-1} \\ \vdots & \ddots & \vdots \\ \text{score}_{n-1,0} & \dots & \text{score}_{n-1,n-1} \end{bmatrix}$$

When computing scores, you should apply two different masks:

- causal mask: if `j > i`, mask that position by setting the score to $-\infty$
- padding mask:
  - if `mask[i] == 0`, the whole score row becomes $-\infty$.
  - if `mask[j] == 0`, mask that position by setting the score to $-\infty$

In C, the score of $-\infty$ will be stored as `-INFINITY` using the `INFINITY` constant defined in `<math.h>`.

Why is the masked score $-\infty$?

We set the score to $-\infty$ because score is the dot product between two vectors, and a small dot product means the vectors are very different. By setting the score to $-\infty$, when we compute the weight in the next stage, these masked values will have weight 0.0.

Why do we use the same mask for both dimensions?

We use the same padding mask for both dimensions, as both dimensions are representing the same token, but for different remixed vectors!

What is the score formula really doing?

If you want the slower derivation, small examples, and a visual story for why the masks are there, use the dot-product explainer and masking explainer.

> ▼ **WORKED EXAMPLE: ONE ATTENTION SCORE**
>
> Lets consider an example with `n = 3` and `d = 2`:
>
> $$\mathbf{Q} = \begin{bmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \\ Q_{2,0} & Q_{2,1} \end{bmatrix} \qquad \mathbf{K} = \begin{bmatrix} K_{0,0} & K_{0,1} \\ K_{1,0} & K_{1,1} \\ K_{2,0} & K_{2,1} \end{bmatrix}$$
>
> Here each row $\vec{\mathbf{Q}}_i$ and $\vec{\mathbf{K}}_j$ of $\mathbf{Q}$ and $\mathbf{K}$ correspond to a vector projected from a token embedding vector. Suppose that the original token embedding vector for rows 0, 1, 2 in $\mathbf{X}$ from the previous part, represented the tokens `Foundations`, `of` and `Algorithms` respectively, this is what the rows of $\mathbf{Q}$ and $\mathbf{K}$ also represent.
>
> For example, if we were calculating $\text{score}_{1,0}$, this is the attention score of query vector 1 `of` with key vector 0 `Algorithms`, or more precisely how much key vector 0 **attends to** query vector 1.
>
> $$\text{score}_{1,0} = \frac{[Q_{1,0} \quad Q_{1,1}] \bullet [K_{0,0} \quad K_{0,1}]}{\sqrt{2}} = \frac{Q_{1,0} \cdot K_{0,0} + Q_{1,1} \cdot K_{0,1}}{\sqrt{2}}$$
>
> Note the same as the previous question, we can also compute the score matrix by consideration of a matrix multiplication (for notational convenience). Here we would need to transpose the second matrix, so we have a defined matrix multiplication where the number of columns of the first matrix must equal number of rows of the second matrix. Mathematically this is equivalent, and how research papers will represent the operation, but you may find one way or another easier to understand when you come to implement it in code.
>
> $$\textbf{score} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} = \frac{1}{\sqrt{d}} \begin{bmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \\ Q_{2,0} & Q_{2,1} \end{bmatrix} \begin{bmatrix} K_{0,0} & K_{1,0} & K_{2,0} \\ K_{0,1} & K_{1,1} & K_{2,1} \end{bmatrix}$$

Stage 4: Attention weights (prompt)

The Stage 3 scores are not the final answer. They must be normalised into weights.

For this stage, you should modify the scaffold function from stage 3 compute_attention_scores_or_weights_prompt(...) so that it can apply softmax.

Hence, when `apply_softmax` is `1` (this stage), you should take the scores and apply stable softmax for each row in the output matrix.

$$\mathbf{weight} = \begin{bmatrix} \text{weight}_{0,0} & \dots & \text{weight}_{0,n-1} \\ \vdots & \ddots & \vdots \\ \text{weight}_{n-1,0} & \dots & \text{weight}_{n-1,n-1} \end{bmatrix} = \begin{bmatrix} \text{softmax}(\overrightarrow{\mathbf{score}}_0) \\ \vdots \\ \text{softmax}(\overrightarrow{\mathbf{score}}_{n-1}) \end{bmatrix}$$

$$\text{weight}_{i,j} = \frac{e^{\text{score}_{i,j} - \max(\overrightarrow{\mathbf{score}}_i)}}{\sum_{u=0}^{n-1} e^{\text{score}_{i,u} - \max(\overrightarrow{\mathbf{score}}_i)}}$$

If a row has no unmasked columns, output `0.0` for each weight in that row, do not divide by zero!

> ▼ **WORKED EXAMPLE: SCORES BECOME WEIGHTS**
>
> If one row of unmasked scores is
>
> ```
> (2, 0, -1)
> ```
>
> then softmax turns that row into weights roughly like
>
> ```
> (0.84, 0.11, 0.05)
> ```
>
> The largest score becomes the largest weight, but every unmasked position still gets a non-negative share. Stable softmax produces the same final weights as ordinary softmax; it just subtracts the row maximum first so the exponentials stay numerically safe.

Stage 5: Attention output (prompt)

Once you have the attention weights, you can compute the actual prompt outputs. Implement this scaffold function: compute_attention_output_prompt(...) .

Compute the attention output vectors as a matrix **output** for each prompt token using:

$$\text{output}_{i,j} = \overrightarrow{\mathbf{weight}}_i \bullet \overrightarrow{(\mathbf{V}^T)}_j = \sum_{u=0}^{n-1} \text{weight}_{i,u} \, V_{u,j} \qquad \mathbf{output} = (\mathbf{weight})\mathbf{V} = \begin{bmatrix} \text{output}_{0,0} & \dots & \text{output}_{0,d-1} \\ \vdots & \ddots & \vdots \\ \text{output}_{n-1,0} & \dots & \text{output}_{n-1,d-1} \end{bmatrix}$$

Note that $\overrightarrow{(\mathbf{V}^T)}_j$ just means, take the vector corresponding to column j of matrix $\mathbf{V}$. Writing $\overrightarrow{\mathbf{V}}_j$ would refer to row j. Doing the transpose first, turns the columns into rows, so we can extract row j to get our vector corresponding to the column.

This is a weighted sum of value vectors using the attention weights you just computed.

▼ **WORKED EXAMPLE: THREE TOKENS EMBEDDED IN TWO DIMENSIONS**

Output example with $n = 3$ and $d = 2$

$$\mathbf{weight} = \begin{bmatrix} \text{weight}_{0,0} & \text{weight}_{0,1} & \text{weight}_{0,2} \\ \text{weight}_{1,0} & \text{weight}_{1,1} & \text{weight}_{1,2} \\ \text{weight}_{2,0} & \text{weight}_{2,1} & \text{weight}_{2,2} \end{bmatrix} \qquad \mathbf{V} = \begin{bmatrix} V_{0,0} & V_{0,1} \\ V_{1,0} & V_{1,1} \\ V_{2,0} & V_{2,1} \end{bmatrix}$$

Using the same example as before, suppose that the original token embedding vector for row 0, 1, 2 in $\mathbf{X}$ represented the tokens `Foundations`, `of` and `Algorithms`. The **weight** matrix would be 3 rows by 3 columns, and the value $\mathbf{V}$ matrix would be 3 rows by 2 columns.

The output vector for token 2, which incorporates the context of the prompt tokens preceding it directly in the embedding would be:

$$\begin{aligned} \overrightarrow{\mathbf{output}}_2 &= \left(\begin{bmatrix} \text{weight}_{2,0} & \text{weight}_{2,1} & \text{weight}_{2,2} \end{bmatrix} \begin{bmatrix} V_{0,0} \\ V_{1,0} \\ V_{2,0} \end{bmatrix} \quad \begin{bmatrix} \text{weight}_{2,0} & \text{weight}_{2,1} & \text{weight}_{2,2} \end{bmatrix} \begin{bmatrix} V_{0,1} \\ V_{1,1} \\ V_{2,1} \end{bmatrix} \right) \\ &= \big((\text{weight}_{2,0} V_{0,0} + \text{weight}_{2,1} V_{1,0} + \text{weight}_{2,2} V_{2,0}) \quad (\text{weight}_{2,0} V_{0,1} + \text{weight}_{2,1} V_{1,1} + \text{weight}_{2,2} V_{2,1}) \big) \end{aligned}$$

If one weight is near 1, the output will look very similar to that one value vector, if weights are spread out, the output is a blend.

Much like the previous parts, we can also write this computation very compactly using a matrix multiplication, which will give us a matrix where each row corresponds to the output vector for a given token

$$\mathbf{output} = (\mathbf{weight})\mathbf{V} = \begin{bmatrix} \text{weight}_{0,0} & \text{weight}_{0,1} & \text{weight}_{0,2} \\ \text{weight}_{1,0} & \text{weight}_{1,1} & \text{weight}_{1,2} \\ \text{weight}_{2,0} & \text{weight}_{2,1} & \text{weight}_{2,2} \end{bmatrix} \begin{bmatrix} V_{0,0} & V_{0,1} \\ V_{1,0} & V_{1,1} \\ V_{2,0} & V_{2,1} \end{bmatrix}$$

How should I interpret the output vector?

This is the point of attention: one prompt token's output vector becomes a blend of value vectors based on which key vectors attended to a given query vector the most.

If one weight dominates, the output will look very similar to one value vector. If several weights matter, the output becomes a mixture of several vectors. The attention explainer shows this more visually.

Stage 6: Generated outputs with a KV cache

In this task you will be using the prompt and the generated tokens (provided as part of the input) and a KV cache to compute attention outputs. You will *not* be generating the tokens as these are provided.

How does this link to a full LLM?

In a full LLM what would happen would be we would first compute attention for all the prompt tokens. Then we would pass this along to other components inside the LLM which will use it to generate the next token. Once we have this new generated token we again need to compute attention, and pass it along, so that a new token can be generated, repeating this as many times as desired. Hence, while we have given all the generated tokens already as input to your code, in a real system, we would only get one at a time.

The final stage applies the same attention logic during generation, but now the program reuses past keys and values through the cache.

Implement this scaffold function: compute_generation_with_cache(...) .

For each generated token t which follows n prompt tokens, you compute:

1. $\vec{\mathbf{Q}}_{n+t}$: the query vector for the token numbered $n+t$.
2. $\vec{\mathbf{K}}_{n+t}$ and $\vec{\mathbf{V}}_{n+t}$: the key and value vectors for the token numbered $n+t$, which should be appended to the KV cache.
3. score the new query $\vec{\mathbf{Q}}_{n+t}$ against all cached keys $\vec{\mathbf{K}}_i$ for $0 \leq i \leq n+t$, remembering to mask any prompt cache entries ($0 \leq i < n$) where `mask[i] == 0` (same as one row of stage 3)
4. apply stable softmax over the cached scores to turn them into weights (same as one row of stage 4)
5. compute the output as a weighted sum of cached values (same as one row of stage 5)

There is no separate future-token causal mask in Stage 6 because the cache only contains past items plus the current generated token. The current generated token is included, so it can attend to itself.

Dot Products Computed

You must also report the exact number of dot products computed for each generated token.

In this stage, define "a dot product computed", as computing the dot product between any two vectors (which includes any matrix multiplication between a given row and column of a matrix), for any weight matrix, and any unmasked prompt or any generated token embedding or projection vectors.

That means:

- For each generated token, you should compute each unmasked score **exactly once** (store it in an array, reuse it for both max and denominator).
- The total count you return should equal the total number of unmasked cached key positions processed across all generated tokens.
- If a prompt cache entry is masked (`mask[j] == 0`), it contributes 0 to the count.

> ▼ **WORKED EXAMPLE: STAGE 6 DOT-PRODUCT COUNTS**
>
> In the visible `test0.txt` case, the prompt mask is `1 1 0`, `d = 2`, and there are two generated tokens (`g = 2`).
>
> - `3d = 6` dot products compute the new query, key, and value vectors
> - `3` dot products score against unmasked cached keys
> - `d = 2` dot products compute the output vector
>
> So the printed count is `11`.
>
> **For the second generated token:**
>
> - `6` dot products compute the new query, key, and value vectors
> - `4` dot products score against unmasked cached keys
> - `2` dot products compute the output vector
>
> So the printed count is `12`.

Why does the cache help here?

The cache exists so you do not keep recomputing old keys and values during generation. The KV-cache explainer shows the cache growth step by step and explains how the per-generated-token dot-product count should be interpreted.

> ### Want A Concrete End-To-End Example?
>
> Now that you have seen the whole pipeline, the Toy demo is a good way to make it concrete.
>
> It lets you generate a small numeric input from real text, run your program on it, and then read back a token-level explanation of the resulting attention weights. It is optional, but it may help you connect the stages to something more human-readable.
>
> The structured version also uses a small one-hot encoding region for entity identity, which is why some of the toy vectors look especially interpretable.

Testing, Marking, and Output Rules

This section collects the parts of the spec that are directly useful while checking your work: testing, common failure modes, marking, and the exact output contract.

Marking and feedback

This assignment has:

- an autograded testcase component
- human-marked components for code style and code structure/approach
- a short written complexity analysis

The stage breakdown in the handout is:

STAGE	TESTCASES	STYLE	STRUCTURE / APPROACH	COMPLEXITY ANALYSIS	TOTAL
1	0.5	0.5	0.5	0.5	2
2	1	0.5	1	0.5	3
3	2	0.5	1	0.5	4
4	1	0.5	1	0.5	3
5	2	0.5	1	0.5	4
6	2	0.5	1	0.5	4

> **Code style expectations**
>
> - define constants before use instead of relying on magic numbers or strings
> - keep `#define` names uppercase
> - include function prototypes for all functions
> - use descriptive names for variables and functions
> - comment on the approach behind sections of code without restating what is clear from the syntax
> - keep bracket placement, indentation, and spacing consistent
> - use blank lines to separate sections and improve readability
> - keep lines of code below 100 characters

> **Code structure expectations**
>
> - avoid duplicated code segments
> - avoid unnecessary global variables by passing data into functions
> - use helper functions where that improves structure
> - keep functions reasonably short and not overly complex
> - choose a straightforward algorithmic approach
> - avoid unnecessary duplication of data
> - avoid deeply nested control flow where possible

> **Complexity Analysis**
>
> Fill in the `Complexity Analysis` block near the top of `a1.c`.
>
> In plain English, give the Big-O time complexity in terms of `n`, `d`, and `g` for:
>
> - Stage 1 create embeddings
> - Stage 2 projections
> - Stage 3 prompt attention scores
> - Stage 4 prompt attention weights
> - Stage 5 prompt attention outputs
> - Stage 6 generation with cache
>
> It is enough to count dominant loop work and ignore constant factors. You do not need to analyse memory usage. The important part is not just stating the Big-O, but briefly explaining how you got it. Analyse the code you actually wrote, not a more optimised version you could have written instead.

Testing your code

Use these sample tests to check your own `a1.c` as you build it.

Inside the provided bundle you have:

- `test_data/test0.txt` with expected output `test_data/test0-out.txt`
- `test_data/test1.txt` with expected output `test_data/test1-out.txt`
- `test_data/test2.txt` with expected output `test_data/test2-out.txt`
- `test_data/test3.txt` with expected output `test_data/test3-out.txt` for a larger end-to-end practice case

Compile:

```
clang -Wall -Wextra -Wno-newline-eof -Wno-unused-parameter -pedantic -std=c17
-lm -o a1 a1.c
```

Run a test:

```
./a1 < test_data/test0.txt
```

Compare output:

```
./a1 < test_data/test0.txt > myout.txt
diff -u test_data/test0-out.txt myout.txt
```

Common pitfalls

- Forgetting the causal mask in Stages 3 to 5. For row `i`, positions `j > i` must be masked.
- Forgetting the padding mask in Stages 3 to 5.
- Forgetting that if `mask[i] == 0`, the whole position `i` should contribute a zero vector in the later stages.
- Not using stable softmax.
- Dividing by zero when all columns are masked.
- In Stage 6, counting dot products incorrectly.

Exact output format

You must print exactly the following stage headers and data, in order:

- Stage 1: one-hot embeddings for the sorted unique tokens
- Stage 2: prompt `Q`, `K`, and `V` projections, each an `n x d` table
- Stage 3: attention scores for the prompt, an `n x n` table
- Stage 4: attention weights for the prompt, an `n x n` table
- Stage 5: attention outputs for the prompt, an `n x d` table
- Stage 6: one generated-output line and one dot-product-count line for each generated token

The exact stage header lines are:

```
Stage 1: Create Embeddings
Stage 2: Projections
Stage 3: Attention Scores (Prompt)
Stage 4: Attention Weights (Prompt)
Stage 5: Attention Output (Prompt)
Stage 6: Generated Outputs
```

The Stage 2 block format is:

```
Q Projection:
<n lines of d numbers>
K Projection:
<n lines of d numbers>
V Projection:
<n lines of d numbers>
```

The Stage 6 line format is:

```
Gen 0: <d numbers>
Dot products computed: <an integer>
Gen 1: <d numbers>
Dot products computed: <an integer>
...
```

Numbers must be printed with exactly three digits after the decimal point, separated by a single space. There must be no extra spaces at line ends and no extra debug output anywhere.

The scaffold's printing functions also clamp extremely small values to `0.000`. That is part of the expected output behaviour, so it is safest to leave the provided printing code alone.

SECTION

FURTHER READING

Tokens and Embeddings

How text becomes ordered rows of numbers before attention begins.

LLMs do not work directly on text. They work on tokens , which are pieces of text in a fixed order, and on embeddings , which are vectors of numbers attached to those tokens.

Text
the
small
cat

→

Tokens in order
0 the
1 small
2 cat

→

Embedding rows
i = 0 `(0.41, -0.12, 0.87, -0.33)`
i = 1 `(0.05, 0.62, -0.14, 0.48)`
i = 2 `(-0.36, 0.91, 0.11, -0.07)`

The order is preserved all the way through: token 0 becomes position `i = 0`, token 1 becomes `i = 1`, and so on. Attention later uses those positions when it decides what each position may look back at.

For this assignment, you can treat tokens almost like words:

- token 0 comes first
- token 1 comes next
- later positions may only look back, not forward

In this assignment, you first practise the representation idea with a simplified embedding step.

Stage 1 builds one-hot vectors for the tokens as a simple stand-in for embeddings. That is the representation exercise students do directly. After that, the later attention stages use richer embedding rows as the prompt matrix that gets projected into $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$.

The prompt matrix can be read as:

- n rows
- one row per prompt token
- d numbers in each row

Prompt matrix shape

n rows by d columns

Rows track token positions. Columns are the components of each embedding vector .

d_0 d_1 d_2 d_3
token 0 0.41 -0.12 0.87 -0.33 token 1 0.05 0.62 -0.14 0.48 token 2 -0.36 0.91 0.11 -0.07

This is why the scaffold naturally uses a 2D array: one axis for token positions and one axis for embedding components.

That is why the rest of the assignment is mostly about array loops and matrix-style computations rather than text processing.

Why the spec says token instead of word

Models do not literally operate on words. A token might be:

- a whole word
- part of a word
- punctuation
- even whitespace in some systems

For this assignment, you can safely pretend tokens are words because the important fact is just that token positions have an order and the attention rules depend on that order.

SECTION

REQUIRED CONCEPT

One-Hot Encoding

How a single active slot can represent a category or label.

One-hot encoding is a way to represent one choice from a small set using a vector of zeros with exactly one `1`.

EXAMPLE MAPPING

The position of the `1` is the identity. Everything else stays `0`.

CATEGORY	SLOT 0	SLOT 1	SLOT 2	SLOT 3
CAT	1	0	0	0
DOG	0	1	0	0
BIRD	0	0	1	0
FISH	0	0	0	1

One-hot encoding gives each category its own slot. The vector is sparse on purpose: the category is encoded by where the `1` appears, not by the size of the number.

For example, if you had three categories, you could write them as:

- `cat` -> `(1, 0, 0)`
- `dog` -> `(0, 1, 0)`
- `bird` -> `(0, 0, 1)`

The important idea is that the position of the `1` carries the identity. The vector is not trying to be smooth or semantic. It is just a direct numeric label.

In a model, one-hot vectors are often used when the system needs a discrete choice in numeric form. They are a simple bridge between categories and vectors, and they are easy to read because only one slot is active.

One-hot vector versus embedding

A one-hot vector is a very sparse representation . It says “this item is category `k`” and leaves the rest of the slots empty.

An embedding is different: it usually has many non-zero values and can encode richer relationships between items. In that sense, a one-hot vector is the simplest possible kind of vector representation.

Why the toy generator mentions entity one-hot slots

The toy input generator uses a small entity one-hot region in its structured mode so repeated noun-like mentions can share a simple identity slot. That is a debugging aid, not something you have to implement in the assignment itself.

If you want the broader “tokens become vectors” picture, read Tokens and Embeddings.

SECTION

REQUIRED CONCEPT

Q, K, and V Projections

Why the same embedding is remixed into query, key, and value vectors.

Each token embedding row is turned into three related vectors:

- **Query ($\vec{\mathbf{Q}}$)**: what this token is looking for
- **Key ($\vec{\mathbf{K}}$)**: what this token offers to be matched against
- **Value ($\vec{\mathbf{V}}$)**: the information that gets blended into the output

The assignment calls this a **projection** because each new vector is produced by multiplying the original embedding row by a different matrix:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_{\mathbf{q}}, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_{\mathbf{k}}, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_{\mathbf{v}}$$

ONE EMBEDDING ROW BECOMES Q, K, AND V

One embedding row x	→	Multiply by `Wq`	→	Query row
	→	Multiply by `Wk`	→	Key row
	→	Multiply by `Wv`	→	Value row

Here is one small worked example.

WORKED PROJECTION EXAMPLE

Input vector `x`

```
(2, 3)
```

One token embedding row.

×

Weight matrix `W`

```
[ [ 1, 10 ],
  [ 0, -1 ] ]
```

Each output component uses one column of `W`.

=

Projected vector `y`

```
(2, 17)
```

The same input row produces both output components.

First output component

`y_0 = 2·1 + 3·0 = 2`

Use the first column of `W`, multiply it entry-by-entry against `x`, then add the results.

Second output component

`y_1 = 2·10 + 3·(-1) = 17`

Use the second column of `W`, multiply it entry-by-entry against the same input row, then add the results.

Why three versions of the same token? Because attention needs one representation for matching, one for being matched against, and one for the information actually carried forward.

SECTION

REQUIRED CONCEPT

Dot Products and Scores

The score computation used to compare one token with another.

A dot product is “multiply matching components and add the results”.

If:

$$\vec{\mathbf{a}} = (1 \quad 2), \qquad \vec{\mathbf{b}} = (3 \quad 4)$$

then:

$$\vec{\mathbf{a}} \bullet \vec{\mathbf{b}} = 1 \cdot 3 + 2 \cdot 4 = 11$$

In attention, the dot product compares the query vector at position `i` with the key vector at position `j`. The assignment uses the **scaled** dot product:

$$\text{score}_{i,j} = \frac{\vec{\mathbf{Q}}_i \bullet \vec{\mathbf{K}}_j}{\sqrt{d}}$$

The scale factor keeps the numbers in a more manageable range before softmax.

This is why Stage 3 is the “score matrix” stage: every allowed pair `(i, j)` gets one score, and masked pairs are stored as `-INFINITY` instead.

How to read the score formula indices

The longer spec also unpacked the indices:

- `i` is the current query position
- `j` is the key position being compared against
- `t` is the vector component index

So:

$$\sum_{t=0}^{d-1} Q_{i,t} K_{j,t}$$

just means “loop over the components, multiply matching entries, and add them”.

Why a dot product is useful here

The dot product acts like a similarity score:

- if two vectors point in similar directions, the dot product tends to be larger

- if they are less aligned, it tends to be smaller

That is the intuition behind why Stage 3 scores later become attention weights.

SECTION

REQUIRED CONCEPT

Attention

How each token decides which earlier tokens matter most.

Attention is the mechanism that lets one token position “look back” at other positions and decide which ones matter for the next computation.

At the simplest level, attention does three things:

1. compare one token with earlier tokens
2. turn those relevance scores into weights
3. use those weights to combine information from the earlier tokens

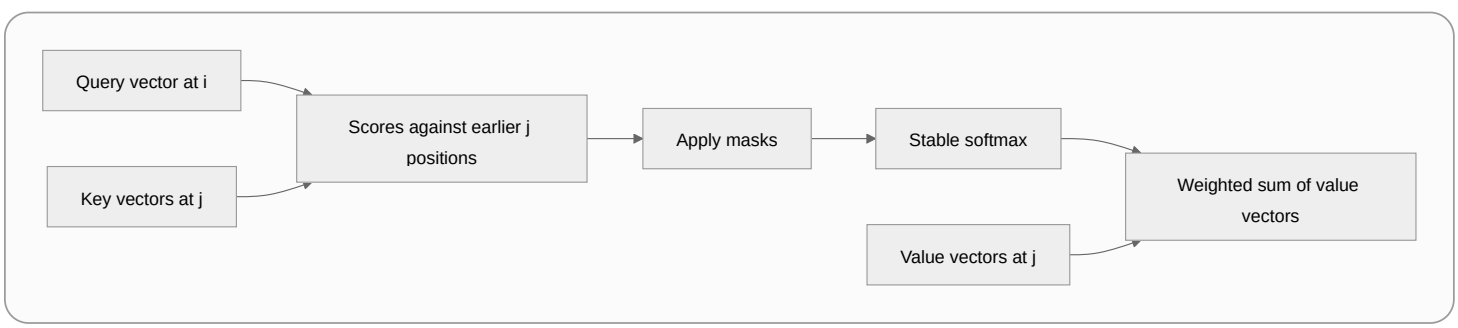
In the assignment, those three steps show up as:

- scores
- weights
- outputs

In this assignment, you implement that pipeline directly:

$$\text{scores} \rightarrow \text{masks} \rightarrow \text{softmax weights} \rightarrow \text{weighted sums}$$

Using the assignment names, one position supplies a query vector $\vec{\mathbf{Q}}_i$, the earlier positions supply key and value vectors $\vec{\mathbf{K}}_j$ and $\vec{\mathbf{V}}_j$, and attention decides how much of each value vector should contribute to the output.



The important simplifications are:

- there is only one attention head
- you are given the embeddings and the projection matrices
- you do not predict the next English token
- you only implement the attention block itself

Modern chatbots generate text one token at a time. At each step the model has to decide which earlier tokens matter most. That “look back and focus” behaviour is attention.

For example:

> Alice gave Bob the book because ___ was late.

The model needs a way to look back over earlier positions and decide which ones are relevant when forming the next internal representation. Attention is the mechanism that does that.

If you want the ingredients that feed attention, start with Q, K, and V projections. If you want the “why do some positions disappear?” rule, read masking.

SECTION

REQUIRED CONCEPT

Causal and Padding Masks

The rules that decide which positions are ignored and why.

Masking means “treat this position as if it is not available”.

The prompt stages use two masks at the same time.

Causal mask

When an LLM is generating text left-to-right, it must not use information from future tokens for a given query token. So when computing attention score for the query vector corresponding to prompt token i, the key vectors corresponding to prompt token j where $j > i$ of $\mathbf{K}$ are **masked out** (forced to have weight `0.0` in the stage 4).

This is called **causal** (or **autoregressive**) self-attention. For prompt position `i`, positions with `j > i` are masked. This stops the model from “looking into the future”.

That is why prompt attention only looks backward or at the current token.

Padding mask

Usually we store a prompt in a fixed-size array, but not every slot is actually used (eg sentences have different lengths). The unused slots are called **padding**. A **padding mask** tells you which prompt positions are real (`1`) vs padding (`0`). If we did not have the padding mask, the padding would be “considered” as having important meaning, and our LLM would output a bunch of padding between other tokens. Not ideal!

THE FOLLOWING DEMO IS UNDER CONSTRUCTION.

It may or may not fully work, or have inconsistent math notation!

PROMPT MASKING WORKED EXAMPLE

Prompt mask: `[1, 1, 0]`

Inspect position: `i = 1`

1. Raw score row

J	0	1	2
I = 1	score_{1,0}	score_{1,1}	score_{1,2}

All three positions are still visible before masking.

→

2. After causal mask

J	0	1	2
I = 1	score_{1,0}	score_{1,1}	future

`j = 2` is blocked because `2 > 1`.

→

3. After padding mask

J	0	1	2
I = 1	score_{1,0}	score_{1,1}	padding

Final usable positions for `i = 1`: `j = 0` and `j = 1`.

If the query position itself is padding

`mask[2] = 0` then `score_{2,j} -> -INFINITY for all j`

At the score stage, a padded prompt position is fully masked. In the later weight/output stages, that then becomes a vector of zeros.

SECTION

REQUIRED CONCEPT

Softmax and Stable Softmax

How raw scores become row-wise weights that sum to one.

Softmax

Scores can be negative or positive, and they do not automatically add up to anything useful. We want weights that:

- are **not negative**, and
- **sum to 1** (so we can interpret them like percentages).

Softmax is the function that does this to create our weights. You can think of softmax as "turn scores into percentages", where the percentages in any given row of the attention weight matrix sum to `100%`. Hence softmax works on a row vector of our **score** matrix to convert this into a new weight vector.

$$\mathbf{unstable_weight} = \begin{bmatrix} \text{unstable_weight}_{0,0} & \dots & \text{unstable_weight}_{0,n-1} \\ \vdots & \ddots & \vdots \\ \text{unstable_weight}_{n-1,0} & \dots & \text{unstable_weight}_{n-1,n-1} \end{bmatrix} = \begin{bmatrix} \text{unstable_softmax}(\overrightarrow{\mathbf{score}}_0) \\ \vdots \\ \text{unstable_softmax}(\overrightarrow{\mathbf{score}}_{n-1}) \end{bmatrix}$$

$$\text{unstable_weight}_{i,j} = \frac{e^{\text{score}_{i,j}}}{\sum_{u=0}^{n-1} e^{\text{score}_{i,u}}}$$

Here, e is the mathematical constant $e \approx 2.71828$, the base of the natural logarithm.

Stable Softmax

The function e^x grows extremely quickly. In C, if scores are large, `exp(score)` can overflow (become infinity).

To avoid this, we can modify our exponential by subtracting the maximum score in the row:

$$e^{\text{score}_{i,j}} \to e^{\text{score}_{i,j} - \max(\overrightarrow{\mathbf{score}}_i)}$$

This ensures that the exponential ranges over $(0, 1]$, because the exponent will be less or equal to zero. Hence our **weight** matrix using stable softmax is

$$\mathbf{weight} = \begin{bmatrix} \text{weight}_{0,0} & \dots & \text{weight}_{0,n-1} \\ \vdots & \ddots & \vdots \\ \text{weight}_{n-1,0} & \dots & \text{weight}_{n-1,n-1} \end{bmatrix} = \begin{bmatrix} \text{softmax}(\overrightarrow{\mathbf{score}}_0) \\ \vdots \\ \text{softmax}(\overrightarrow{\mathbf{score}}_{n-1}) \end{bmatrix}$$

$$\text{weight}_{i,j} = \frac{e^{\text{score}_{i,j} - \max(\overrightarrow{\mathbf{score}}_i)}}{\sum_{u=0}^{n-1} e^{\text{score}_{i,u} - \max(\overrightarrow{\mathbf{score}}_i)}}$$

For this assignment you compute softmax using `exp()` from `<math.h>`.

Softmax is easiest to remember as **turning scores into percentages**.

STABLE SOFTMAX WORKED EXAMPLE

1. Start with one score row

```
scores = (2, 0, -1)
```

A has the largest score, so it starts with the biggest advantage.

2. Shift by the row maximum

```
max = 2
(2, 0, -1) → (0, -2, -3)
```

Subtracting the same maximum from every entry keeps the final weights the same, but makes the exponentials numerically safer.

3. Exponentiate the shifted row

$(e^0, e^{-2}, e^{-3}) \rightarrow (1.00, 0.14, 0.05)$

The larger original score still turns into the largest exponential term.

4. Normalise into weights

```
sum = 1.00 + 0.14 + 0.05 = 1.19
weights = (1.00, 0.14, 0.05) / 1.19
weights ≈ (0.84, 0.11, 0.05)
```

Add the Step 3 values to get the denominator, then divide each Step 3 value by that same denominator. The final row is non-negative and sums to 1.

What stable softmax changes

Only the intermediate exponentials change. The normalised weights stay the same.

Masking rule still matters

Only unmasked positions belong in the maximum search and the denominator. If nothing is unmasked, the whole row is zeros.

Two implementation details matter:

- only unmasked positions belong in the maximum search and denominator
- if a row has no unmasked positions, the whole output row is zeros

Softmax uses the exponential function because it has helpful properties: it is always positive, it grows smoothly with the score (so bigger scores get bigger weights), and it interacts nicely with calculus during training (you are not doing training).

SECTION

REQUIRED CONCEPT

KV Cache

Why generation can reuse past keys and values instead of recomputing them.

The KV cache is a saved list of past key and value vectors .

Without it, generation would keep recomputing `K` and `V` for every earlier token at every step. That would repeat work unnecessarily.

With a cache, each generated step only needs to:

1. compute the new token's $\vec{\mathbf{Q}}_{n+t}$, $\vec{\mathbf{K}}_{n+t}$, and $\vec{\mathbf{V}}_{n+t}$
2. append the new key and value vectors to the cache
3. score the new query vector against the cached keys
4. build the weighted sum from the cached values

THE FOLLOWING DEMO IS UNDER CONSTRUCTION.

It may or may not fully work, or have inconsistent math notation!

KV CACHE WORKED EXAMPLE

1. Fresh work for the new token

```
h₂ · WQ → q₂
h₂ · WK → k₂
h₂ · WV → v₂
```

For each generated token, you still compute one fresh query, key, and value.

2. Append only the new key and value

```
keys:   [k₀, k₁]   →   [k₀, k₁, k₂]
values: [v₀, v₁]   →   [v₀, v₁, v₂]
```

Earlier cache entries stay exactly as they were.

3. Score the new query against cached keys

```
q₂ · [k₀, k₁, k₂] → [s₀, s₁, s₂]
```

The new query compares against the whole stored history, including the just-appended key.

4. Use those weights on cached values

```
[w₀, w₁, w₂] × [v₀, v₁, v₂] → output₂
```

The output is built from cached values, not from recomputing older token states.

5. The next token reuses the same cache

```
h₃ · WQ → q₃
q₃ · [k₀, k₁, k₂] → scores
weights × [v₀, v₁, v₂] → output₃
```

The cache does not make $\mathbf{q}_3$ free. It saves recomputing the older k and v entries.

That is why cached generation is “one new query against an ever-growing history” rather than “recompute everything from scratch”.

Why no-cache generation recomputes old K and V

Without a cache, the model does not get to say “just compute $\vec{\mathbf{q}}_3$”.

At the next generation step, the input prefix is now the whole sequence so far. A plain Transformer forward pass processes that whole prefix again, which means it rebuilds the older positions’ activations too. So old k and v vectors such as $\vec{\mathbf{k}}_3$ and $\vec{\mathbf{v}}_2$ would be recomputed along with the new token’s work.

With a KV cache, those older $\vec{\mathbf{K}}/\vec{\mathbf{V}}$ vectors are already stored. The model still computes the new token’s fresh projections, but it reuses the saved history instead of rebuilding it.

Why the cache exists at all

The cache is best understood as “saving work you already did”.

Prompt keys and values do not change while generation continues. So once they exist, you can keep them in a cache and only compute the new generated token’s $\vec{\mathbf{K}}$ and $\vec{\mathbf{V}}$.

Useful support appendices

These appendices are directly useful while implementing or checking the assignment. They are kept separate from the broader optional background reading.

COMP10002 FOUNDATIONS OF ALGORITHMS

Annotated Scaffold

A guided reading of the provided `a1.c`, with direct links from the main spec and tooltip-style explanations for the names, arrays, and helper functions you keep seeing.

Overview

This page is meant to be open beside `a1.c`.

The main assignment page tells you **what** to implement. This page is about **how to read the scaffold you were given** without getting lost in the arrays, the stage order, or the helper names.

> **Why this page exists**
>
> The scaffold does some important work for you already:
>
> - it allocates the main arrays
> - it calls the stages in order
> - it prints the stage headers and formatted rows
> - it provides the printing helpers you should normally leave alone
>
> That means your job is mostly to fill in the TODO functions correctly and keep the existing output contract intact.

File map

TOP OF FILE

Comment block, includes, and constants

The header identifies the file, the complexity-analysis block is where students write their Big-O notes, and the preprocessor section defines the fixed array bounds such as MAX_TOKENS , MAX_D , and MAX_GEN .

INTERFACE

Function prototypes

This section tells you the shape of every function before you reach the implementations. When you are coding a stage from the main page, the prototype is the fastest way to remind yourself which arrays are inputs and which arrays you are expected to fill.

EXECUTION FLOW

`main()` is the roadmap

The `main` function is worth reading carefully once, even if you do not edit it much. It shows the exact order in which the scaffold expects the stages to happen, which arrays are reused later, and where the printing already happens.

YOUR CODE

TODO functions

The core work lives in the TODO block: `read_input`, `create_embeddings`, `compute_projection`, the combined prompt score/weight function, the prompt output function, and the generation-with-cache function. Those are the places you should spend most of your implementation effort.

LEAVE THESE ALONE

Provided helpers

The printing helpers already implement the expected formatting, including clamping near-zero values. This is one of the easiest places to accidentally break the autograder if you start “cleaning things up”.

Walking Through `main()`

The easiest way to understand the scaffold is to read `main()` as a pipeline:

```
 /* Some details of the main function such as printing and variable declarations
    are ommitted here for brevity, but are included in the skeleton code */
int main(void) {
    /* allocate all the main arrays */
    read_input(&n, &d, &g, &text_len, embedding_table, mask, prompt, gen, wq, wk, wv);

    /* Stage 1: sort and print the one-hot table */
    create_embeddings(embedding_table, text_len);

    /* build Q, K, V */
```

```
    compute_projection(n, d, prompt, wq, q_matrix);
    compute_projection(n, d, prompt, wk, k_matrix);
    compute_projection(n, d, prompt, wv, v_matrix);

    /* scores -> weights -> outputs */
    compute_attention_scores_or_weights_prompt(
        n, d, mask, q_matrix, k_matrix, scores, NO_SOFTMAX);
    compute_attention_scores_or_weights_prompt(
        n, d, mask, q_matrix, k_matrix, weights, APPLY_SOFTMAX);
    compute_attention_output_prompt(n, d, weights, v_matrix, out_prompt);

    /* generation with cache */
    compute_generation_with_cache(..., t, ..., output);
}
```

What to notice first

The scaffold already decides the stage order for you. If your output is wrong, one of the earlier filled arrays is usually the real cause.

Why there are separate arrays

The scaffold does not overwrite the prompt matrix in place. It keeps `prompt`, then separate `q_matrix`, `k_matrix`, `v_matrix`, then separate `scores`, `weights`, and `out_prompt` so each stage can be inspected independently.

What printing means for you

Notice where `printf` already happens. That is why the spec keeps warning you not to add extra output and not to change the existing formatting helpers.

`read_input(...)`

STAGE 0

Load the whole snapshot into memory

This function is the bridge between the input file and every later computation. If this stage is wrong, nothing downstream is trustworthy.

```
void read_input(int *n, int *d, int *g, int *text_len,
                char embedding_table[MAX_TEXT_SIZE][MAX_TOKEN_LENGTH + 1],
                int mask[MAX_TOKENS],
                double prompt[MAX_TOKENS][MAX_D],
                double gen[MAX_GEN][MAX_D],
                double wq[MAX_D][MAX_D], double wk[MAX_D][MAX_D],
                double wv[MAX_D][MAX_D]);
```

n | d | g | text_len | embedding_table | mask | prompt | gen | Wq / Wk / Wv

What this function should do

Use `scanf` to read the values in the exact order the spec gives: sizes, Stage 1 token text, prompt mask, prompt rows, generated rows, then `Wq`, `Wk`, and `Wv`. This is a storage stage, not a computation stage.

Why the scalar values are pointers

`n`, `d`, `g`, and `text_len` are passed by address so `read_input` can write them back into the variables owned by `main()`.

Common mistake

Mixing up loop bounds. The `prompt` loops run over `n` rows, `gen` runs over `g`, and each matrix `Wq/Wk/Wv` runs over `d × d` entries.

`create_embeddings(...)`

STAGE 1

Turn the token list into a sorted one-hot table

This is the warm-up representation stage. It is separate from the later numeric prompt embeddings, and its whole job is to sort the unique token strings and print the one-hot rows.

```
int compare_tokens(const void *a, const void *b);
void create_embeddings(char embedding_table[MAX_TEXT_SIZE][MAX_TOKEN_LENGTH + 1],
                       int text_len);
```

embedding_table | text_len | compare_tokens

What this function should do

Sort the token strings, keep each unique token once, and print one one-hot vector per token in sorted order.

What the vector length is

The Stage 1 vector length is the number of unique tokens you ended up with after sorting and deduplicating, not the later prompt embedding dimension `d`.

Common mistake

Mixing up “first seen” with the final printed order. First seen helps define the unique set; the printed order is the sorted order.

compute_projection(...)

STAGE 2

Apply the same matrix-multiply pattern three times

This is the reusable projection helper. The scaffold calls it three times so the same loop structure builds `Q`, `K`, and `V`.

```
void compute_projection(int count, int d, double src[MAX_TOKENS][MAX_D],
                        double w[MAX_D][MAX_D],
                        double dest[MAX_TOKENS][MAX_D]);
```

count | src | w | dest

Loop shape

One loop over source rows, one loop over output columns, and an inner accumulation loop over the shared dimension `d`.

Mental model

Each output component is "one source row dotted with one column of the weight matrix". The projection explainer shows that visually.

Common mistake

Forgetting to reset the accumulator to `0.0` for each destination component before the inner loop starts.

compute_attention_scores_or_weights_prompt(...)

STAGES 3 AND 4

Use one function twice: first for scores, then for weights

The scaffold combines raw score computation and weight computation using stable softmax into one function. The first call fills the square prompt score matrix; the second call repeats the same masking logic and turns each row into attention weights. The only reason we compute the scores on their own seperately, is as a check that your program output is correct.

```
void compute_attention_scores_or_weights_prompt(
    int n, int d, const int mask[MAX_TOKENS],
    double q[MAX_TOKENS][MAX_D], double k[MAX_TOKENS][MAX_D],
    double scores_or_weights[MAX_TOKENS][MAX_TOKENS],
    int apply_softmax);
```

q k scores_or_weights apply_softmax

What the first call does

When `apply_softmax == NO_SOFTMAX`, compute the scaled dot products, apply both masks, and store `-INFINITY` in the masked positions. If `mask[i] == 0`, the whole row is masked.

What the second call does

When `apply_softmax == APPLY_SOFTMAX`, use the same allowed positions but turn each row into a stable softmax over the unmasked columns. If a row has no valid columns, the whole output row should be zeros.

Why the output is square

Each prompt row is compared with every prompt row position, so the output is `n × n` even though `q` and `k` are `n × d`.

`compute_attention_output_prompt(...)`

STAGE 5

Use the weights to blend value rows into output rows

This is the weighted-sum stage. The previous stage decided how much each earlier row matters; this stage uses those coefficients to build the new output vector.

```
void compute_attention_output_prompt(
    int n, int d,
    double weights[MAX_TOKENS][MAX_TOKENS],
    double v[MAX_TOKENS][MAX_D], double out[MAX_TOKENS][MAX_D]);
```

weights v out

Why the dimensions fit

A weight row has length `n`, and each column of `v` also has `n` entries. That is why you can think of each output component as “one weight row dotted with one column of `v`”.

What changes conceptually

This is the point where a prompt row stops being “just itself” and becomes a mixture of the value rows from the positions it attended to.

Common mistake

Looping over the wrong dimension in the inner sum. The inner sum here runs over prompt positions `j`, not over embedding components.

compute_generation_with_cache(...)

STAGE 6

Do the same attention logic again, but reuse history

This function is the most crowded because it has to compute the new generated token's projections, append to the caches, score against the cache, produce the output, and return the dot-product count.

```
long compute_generation_with_cache(
    int n, int d, int g, int t, const int mask[MAX_TOKENS],
    double gen[MAX_GEN][MAX_D], double wq[MAX_D][MAX_D],
    double wk[MAX_D][MAX_D], double wv[MAX_D][MAX_D],
    double k_cache[MAX_TOKENS + MAX_GEN][MAX_D],
    double v_cache[MAX_TOKENS + MAX_GEN][MAX_D],
    double output[MAX_D]);
```

t | gen | k_cache | v_cache | output

What makes this different

Prompt attention compares prompt rows to other prompt rows. Generation compares one fresh generated query against the already-built cache of older keys and values.

Why the cache arrays are bigger

The cache has to hold both the original prompt rows and the later generated rows, so the row capacity is `MAX_TOKENS + MAX_GEN`.

What the return value means

The function returns a `long` because the stage also has to report how many dot products were computed for that generated token.

Provided Helpers

These are the functions you should read once, understand, and usually leave alone:

```
static double clamp_near_zero(double value) { ... }
void print_vector(int d, const double row[]) { ... }
void print_attention_matrix(int n,
    const double matrix[MAX_TOKENS][MAX_TOKENS]) { ... }
void print_embedding_matrix(int n, int d,
    const double matrix[MAX_TOKENS][MAX_D]) { ... }
int compare_tokens(const void *a, const void *b) { ... }
```

> **Do not casually rewrite the printing helpers**
>
> Why they matter:
>
> - `clamp_near_zero(...)` rounds tiny floating-point numbers to `0.0`
> - `print_vector(...)` print a length-`d` vector rounded to 3 decimal places seperated by spaces.
> - `print_embedding_matrix(...)` print an n by d embedding matrix with each row on a new line.
> - `print_attention_matrix(...)` print an n by n attention matrix with each row on a new line.
> - `compare_tokens(...)` is an optional helper function to use for sorting an array of strings with qsort
>
> These helpers are part of the expected output behaviour, so changing them is risky unless the spec explicitly tells you to.

COMP10002 FOUNDATIONS OF ALGORITHMS

What An Input File Looks Like

A concrete example of the assignment input format, using a small sample file and a line-by-line explanation.

Raw sample input

Here is the visible `test_data/test0.txt` input file with the same numbers and line breaks as the real file. The colours are added here only to make the sections easier to spot:

`n d g text_len` Stage 1 text mask prompt embeddings generated embeddings Wq Wk Wv

```
3 2 2 9
the subject foundations of algorithms is the best subject
1 1 0
1 0
0 1
1 1
1 1
0.5 -0.5
1 0
0 1
1 0
0 1
1 0
0 1
```

If you only want the big picture, read it like this:

- `3 2 2 9` means `n = 3`, `d = 2`, `g = 2`, and `text_len = 9`
- the next line is the token list used only for Stage 1 embedding creation
- `1 1 0` is the prompt mask
- the next `n = 3` lines are prompt embeddings
- the next `g = 2` lines are generated embeddings
- the last `3d = 6` lines are the matrices `Wq`, `Wk`, and `Wv`, each with `d = 2` rows

Line-by-line breakdown

Each part of the file means:

Line 1

`n d g text_len`

`3 2 2 9` means `n = 3` prompt tokens, `d = 2` components per vector , `g = 2` generated tokens, and `text_len = 9` input tokens for Stage 1.

Line 2

`Stage 1 token list`

This whitespace-separated text is what Stage 1 turns into a sorted unique-token list and then into one-hot embeddings.

Line 3

`mask`

`1 1 0` is the prompt padding mask. Prompt positions `0` and `1` are real, and position `2` is padding.

Lines 4 to 6

`prompt_embeddings`

These are the `3` prompt embedding rows, each of length `d = 2`.

Lines 7 to 8

`generated_embeddings`

These are the `2` generated embedding rows, also each of length `d = 2`.

Lines 9 to 10

`Wq`

These two rows form the query projection matrix `Wq`.

Lines 11 to 12

`Wk`

These two rows form the key projection matrix `Wk`.

Lines 13 to 14

`Wv`

These two rows form the value projection matrix `Wv`.

Two details matter:

- the actual input file does **not** contain labels like `mask` or `Wq`; only this page adds labels and colours to make the structure easier to read
- input is whitespace-separated, so `scanf` does not care whether values are separated by spaces or newlines

If you want a more human-friendly example generated from real text instead of a tiny numeric test case, use the Toy demo.

COMP10002 FOUNDATIONS OF ALGORITHMS

Strings in C

The small amount of C string handling you may need for the one-hot embedding stage: what a string is, how to compare and copy it, and what usually goes wrong.

Overview

If you have not learnt C strings yet, the good news is that you only need a small subset for this assignment.

For the one-hot embedding stage, the main jobs are:

- store tokens as strings
- compare tokens to see whether two strings are the same
- sort strings into lexicographic order
- copy strings into your own arrays when needed

You do not need advanced text processing.

What a string is

In C, a string is a `char` array ending with the special character `'\0'`, called the NUL byte .

For example:

```
char word[] = "cat";
```

This is stored as four characters:

```
'c' 'a' 't' '\0'
```

HOW `CHAR WORD[] = "CAT";` **IS STORED**

The visible word is `cat`, but the array also stores one final byte to mark where the string stops.

INDEX	0	1	2	3
BYTE	'c'	'a'	't'	'\0'
ASCII	99	97	116	0

The final `'\0'` is part of the array. It is the byte that tells C where the string ends.

That final `'\0'` is what tells C where the string ends. It is written as `'\0'` because it is the character with numeric value zero.

This is not the same as NULL . `'\0'` is a byte inside a character array. `NULL` is used for pointers.

So if you store many tokens, a common pattern is:

```
char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH + 1];
```

That means:

- each row stores one token string
- each token can use up to `MAX_TOKEN_LENGTH` visible characters
- one extra slot is needed for `'\0'`

Reading a string using scanf

To read in a string with scanf, we need to specify the maximum size of our char array. For example if stdin is "`Hello World`", we would need to have atleast 6 chars of space allocated in our char array to store the visible characters and the NUL `'/0'` character of "`Hello`".

We could use the array `char str[6]` with `scanf("%5s ", str)` to read `{'H','e','l','l','o','\0'}` into this array. If we tried to read "`Hello! World`", scanf would fail as it would find 6 chars before reaching a whitespace character. To build this format string from the array size constants, the skeleton provides you with the constant `TOKEN_STR_SCANF_FORMAT` which you can use.

Important Warning

All the inbuilt string functions in C assume your char array is NUL `'\0'` terminated. If it isn't, this will cause out of bounds memory access and will likely crash your program or cause weird behaviour. Be careful!

Compare and copy

The most important rule is this:

```
== does not compare string contents
```

In C, `==` on arrays or pointers does not mean “do these two words have the same letters?”. For strings, use functions from `<string.h>`.

Useful ones here are:

```
strcmp(a, b)
```

- returns `0` if the strings are equal
- returns a negative value if `a` comes before `b`
- returns a positive value if `a` comes after `b`

That is why `strcmp` is useful both for:

- checking whether two tokens are the same
- sorting tokens lexicographically

If you want to use the library sort for Stage 1 rather than writing your own sorting loop, see qsort and function pointers in C.

To copy a string into your own array, use:

```
strcpy(dest, src);
```

If you want the length of a string, use:

```
strlen(s);
```

COMP10002 FOUNDATIONS OF ALGORITHMS

Checking And Debugging Your C Code

Practical commands for compiling with strong warnings, checking your code for bugs, and using tools that help you catch mistakes before submission.

Baseline build

Work from the directory that contains your own `a1.c`.

The safest starting point is to compile with the same warning level and language mode used by the assignment:

```
clang -Wall -Wextra -Wno-newline-eof -Wno-unused-parameter \
  -pedantic -std=c17 -o a1 a1.c -lm
```

This is a good default because it is close to what the autograder expects. If your code does not compile cleanly here, fix that first.

Why first-year students should use these tools

If you are new to C, these tools are helpful because many C mistakes do not produce neat, obvious error messages.

A compiler might reject your code completely, but it can also accept code that still has a bug in it. Extra warnings , static analysis , and sanitizers give you more chances to catch problems early.

That matters in first year because:

- small syntax or type mistakes are easier to fix when the compiler points at them directly
- array and loop bugs are common in early C programs
- C is less forgiving than Python, so some mistakes compile but behave badly at runtime
- learning to use tools is part of learning to program, not an optional extra

For this assignment, these tools are especially useful for finding:

- wrong format strings in `printf` or `scanf`
- suspicious loop bounds
- accidental type conversions
- out-of-bounds array access

- undefined behaviour such as invalid shifts or bad memory use

What “static analysis” means

When you compile a C program, the compiler already checks some things for you, such as syntax errors and some suspicious code patterns.

Static analysis means:

- a tool inspects your source code without running the program
- it tries to spot bugs, risky logic, or style problems
- it gives you warnings before you test on actual input

For this assignment, it helps to separate three kinds of tools:

- `Compiler warnings`: messages from `clang` or `gcc` during compilation
- `Static analyzers`: tools such as `clang --analyze`, `gcc -fanalyzer`, or `cppcheck` that inspect code paths more deeply

These tools are useful, but they are not magic. A warning may be a false alarm, and a clean report does not prove the program is correct.

Clang flags

If you want a stricter local Clang build, try:

```
clang -Wall -Wextra -Wshadow -Wconversion -Wdouble-promotion \
  -Wformat=2 -Wstrict-prototypes -pedantic -std=c17 -o a1 a1.c -lm
```

These flags can help catch:

- variable shadowing
- implicit numeric conversions
- format-string mistakes
- missing or weak function declarations

While you are still filling in the scaffold, you may also see unused-parameter warnings from unfinished `TODO` functions. That is normal in incomplete code. Once the functions are implemented, many of those warnings disappear.

Not every warning here must be enabled for grading, but they are useful while cleaning up your code.

Clang also includes a built-in static analyzer:

```
clang --analyze -Wall -Wextra -pedantic -std=c17 a1.c
```

If it reports a possible null dereference , dead store , or suspicious path , inspect it carefully. Some reports are false positives , but many are worth fixing.

GCC flags

On Linux with a real GNU `gcc`, a comparable warning build is:

```
gcc -Wall -Wextra -Wshadow -Wconversion -Wformat=2 \
  -Wstrict-prototypes -pedantic -std=c17 -o a1 a1.c -lm
```

Recent GCC versions also have an analyzer mode:

```
gcc -fanalyzer -Wall -Wextra -pedantic -std=c17 -o a1 a1.c -lm
```

Important note: on many macOS setups, `gcc` is actually Apple Clang in disguise. If `gcc --version` prints something like `Apple clang`, then `-fanalyzer` is not the GNU GCC analyzer and may not exist. In that case, use `clang --analyze` and `cppcheck` instead.

Checking tools

cppcheck

`cppcheck` is available in many lab or Homebrew-style environments and is installed in this local environment. A practical command is:

```
cppcheck --enable=warning,style,performance,portability \
  --std=c17 --language=c --inline-suppr a1.c
```

If you want more detail, add:

```
cppcheck --enable=all --std=c17 --language=c --inline-suppr a1.c
```

`cppcheck` is especially useful for:

- suspicious conditions
- uninitialised variables

- unreachable branches
- portability warnings

It can also be noisy, so treat it as a review tool, not an oracle.

Clang static analyzer

If you are using Clang, this is the simplest built-in static-analysis pass:

```
clang --analyze -Wall -Wextra -pedantic -std=c17 a1.c
```

This is path-sensitive , so it sometimes catches issues that ordinary warning flags miss.

Runtime tools

Strict warnings and static analysis help, but runtime tools catch a different class of bugs.

A sanitizer is a runtime checking tool. You compile your program with extra instrumentation, then run it normally. If something dangerous happens while the program is executing, the sanitizer tries to stop immediately and explain the problem.

The most useful sanitizers for this assignment are:

- `-fsanitize=address` for out-of-bounds access, use-after-free , and related memory bugs
- `-fsanitize=undefined` for undefined behaviour such as bad shifts, signed overflow , or invalid pointer use
- `-fsanitize=leak` for memory leaks, if your solution uses dynamic allocation

AddressSanitizer on its own is already very useful:

```
clang -g -fsanitize=address -fno-omit-frame-pointer \
  -std=c17 -o a1-asan a1.c -lm
./a1-asan < test_data/test0.txt
```

If your program reads or writes outside an array, uses freed memory, or corrupts the stack, AddressSanitizer tends to gives a much better explanation than a normal crash.

A good general-purpose local debug build is:

```
clang -g -fsanitize=address,undefined -fno-omit-frame-pointer \
  -std=c17 -o a1-sanitize a1.c -lm
```

```
./a1-sanitize < test_data/test0.txt
```

If you are using dynamic allocation and specifically want leak checking, you can also try:

```
clang -g -fsanitize=address,undefined,leak -fno-omit-frame-pointer \
  -std=c17 -o a1-sanitize a1.c -lm
```

Suggested workflow

For this assignment, a sensible local checking loop is:

1. Compile with the baseline assignment command.
2. Recompile with a stricter warning set.
3. Run `cppcheck` if available.
4. Run `clang --analyze` if you are using Clang.
5. Run an AddressSanitizer build if something is crashing or behaving strangely.

No single tool is enough by itself. The goal is to combine fast compiler feedback, course-specific style checks, and at least one analyzer that looks for suspicious code paths.

COMP10002 FOUNDATIONS OF ALGORITHMS

Toy Demo

A small end-to-end way to connect the assignment's numeric inputs and outputs back to token positions in real text.

Generate a toy input

From the assignment bundle root:

```
python3 toy_llm_generate_input.py --mode structured --n 6 --g 2 --d 6 \
  --text "the cat sat on the mat and purred softly" \
  --show-tokens > toy-input.txt
```

The `--mode structured` option intentionally makes the matrices and embeddings easier to reason about. It is meant for learning and debugging, not for grading.

The structured generator also uses a small one-hot encoding region for entity identity, so repeated noun-like mentions can share a simple slot.

Then run your program:

```
./a1 < toy-input.txt > toy-output.txt
```

Explain the output

Use the helper script to turn the numeric output back into a token-oriented explanation of the Stage 4 weights:

```
python3 toy_llm_explain_output.py \
  --input toy-input.txt \
  --text "the cat sat on the mat and purred softly" \
  --output toy-output.txt
```

If the explanation does not perfectly match your grammar intuition, that is fine. Attention is computed from the vectors and matrices, not from English rules directly. The goal of the toy demo is to make the output feel less abstract.

Expanded glossary

The main spec already introduces terms in context. This appendix is the expanded printable glossary that collects the fuller definitions in one place.

Callback

A function passed into another function so it can be called when needed.

Causal mask

The mask that stops position i from attending to any future position j > i.

Compiler

A program that translates C source code into an executable and reports errors or warnings it notices.

Context window

The amount of earlier text or tokens a model can look at at once when processing or generating.

Dead store

Assigning a value to a variable that is never used before being overwritten or going out of scope.

Dot product

Multiply matching positions and add them up.

Dynamic allocation

Requesting memory while the program runs, usually with functions such as malloc, calloc, or realloc.

Embedding

A vector representing a token as numbers.

Embedding lookup

The step that turns each token ID into its stored embedding vector.

False positive

A tool report that looks like a bug warning but turns out not to be a real bug in your code.

Format string

The text passed to printf or scanf that tells the function what kinds of values to print or read.

Function declaration

The part of a function definition or prototype that gives the function name, return type, and parameter types.

Function pointer

A value that stores the address of a function so another part of the program can call that function later.

Generate

Produce new tokens one at a time after the prompt.

Implicit numeric conversion

A number changing type automatically, such as from double to int, without an explicit cast.

KV cache

Stored past keys and values used during generation.

Lexicographically

In dictionary order, comparing strings from left to right.

LLM

Large Language Model, a model that processes tokens and generates text one token at a time.

Mask

A rule that says "ignore this position".

Matrix

A table of numbers. In C, a 2D array of double.

NUL byte

The byte with numeric value zero used to mark the end of a C string, usually written as '\0'.

Null dereference

Trying to read or write through a pointer that is NULL.

Padding mask

The mask that ignores prompt positions marked as padding, usually where mask[j] == 0.

Path-sensitive

Looking at different possible branches through the program separately instead of treating them all as one generic flow.

Portability warning

A warning that code may behave differently or fail on another compiler, operating system, or machine.

Prompt

The starting tokens you already have before generating new tokens.

Prompt matrix

The matrix whose rows are the prompt token embeddings, with one row per token and one column per embedding component.

Sanitizer

A runtime checking tool that adds extra checks to your compiled program so bugs are reported while it runs.

Score

A number saying how relevant one token is to another.

Self-attention

A mechanism where each token can look at other tokens in the same sequence and decide how much they should influence its new representation.

Shadowing

Reusing a variable name in an inner scope so it hides another variable with the same name.

Signed overflow

An arithmetic result on a signed integer type that goes past the range that type can represent.

Softmax

A function that turns scores into weights that sum to 1.

Sparse representation

A representation where most entries are zero, so only a small number of positions carry information.

Static analysis

Checking source code for likely bugs without running the program.

Suspicious condition

A test in an if, while, or similar statement that looks wrong, misleading, or unlikely to behave the way the programmer intended.

Suspicious loop bounds

A loop whose start, stop, or increment conditions look wrong, risky, or inconsistent with the array or range it is meant to process.

Suspicious path

A control-flow path that looks inconsistent or risky, often because conditions or assumptions do not line up the way the programmer intended.

Token

A piece of text, sometimes a whole word and sometimes part of a word.

Tokenisation

The step that splits raw text into tokens before the model processes them.

Undefined behaviour

Code whose result is not reliably defined by the C language, so it may behave unpredictably.

Uninitialised variable

A variable that is used before it has been given a known value.

Unreachable branch

A part of the code that can never actually run because the conditions leading to it are impossible.

Use-after-free

Trying to access memory after it has already been released.

Vector

A fixed-length list of numbers. In C, a 1D array of double.

Warning

A message from a tool saying that code looks suspicious or risky, even if it still compiles.

OPTIONAL

Optional background appendices

These appendices are interesting background, context, or extension reading. They are not needed in the same way as the support appendices above.

FURTHER READING

Reading Transformer Paper Notation

How to read the compact notation used in the original Transformer paper and later ones.

The original Transformer paper, *Attention Is All You Need*, and many later papers often write attention in one compact line:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

This notation is compact, but it hides the row-by-row logic that matters for implementation.

On this page the first displayed formula keeps the paper's plain `Q`, `K`, and `V`. Elsewhere in this web spec, the same matrices are written as $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ to match the main TeX handout.

This page is about reading that paper-style notation confidently:

- what the symbols mean
- what matrix products like `QK^T` are doing
- how the compact equation expands into the row operations you would actually code

Here is the direct translation:

- $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are whole matrices, not single vectors
- $\mathbf{Q}\mathbf{K}^T$ means "compute all query-vs-key dot products"
- softmax is applied **row-wise**
- multiplying by $\mathbf{V}$ means "use each row of weights to form a weighted sum of value vectors"

In implementation terms:

- Stage 2 builds $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$
- Stage 3 builds the score matrix
- Stage 4 applies row-wise stable softmax with masking
- Stage 5 multiplies those weights by $\mathbf{V}$

What the compact formula is really compressing

The compact formula hides these matrix meanings:

- $\mathbf{Q}\mathbf{K}^\top$ is the full score matrix
- $\mathrm{softmax}(\cdot)$ is row-wise
- multiplying by $\mathbf{V}$ means "take weighted combinations of the value vectors"

If you are comfortable with the one-line formula but keep forgetting the row direction, remember this:

every row is one token asking, "which earlier rows matter for me?"

What papers often omit in the main equation:

- which direction softmax runs
- the masking rules
- the exact array shapes

That is why a teaching spec is more verbose than the paper formula: the loops need the hidden details made explicit.

How to read the one-line formula as row operations

The compact notation expands into these shapes:

- $\mathbf{Q} \in \mathbb{R}^{n \times d}$ means each row is one query vector
- $\mathbf{K} \in \mathbb{R}^{n \times d}$ means each row is one key vector
- $\mathbf{V} \in \mathbb{R}^{n \times d}$ means each row is one value vector

Then:

- $S = \mathbf{Q}\mathbf{K}^\top$ is the score matrix
- $A = \mathrm{softmax}(S)$ is the row-wise weight matrix
- $O = A\mathbf{V}$ is the output matrix

Why papers suddenly introduce extra letters

When you read later Transformer papers, you may also see:

- batch dimensions like `(B, n, d)`
- head indices for multi-head attention
- compact summation notations that suppress explicit `\sum` symbols

Those are extensions of the same computation, not fundamentally different algorithms.

COMP10002 FOUNDATIONS OF ALGORITHMS

Transformer Explainer

A plain-English explanation of what a Transformer is, why attention sits at its core, and where this assignment fits into the larger model.

Short answer

A Transformer is a neural-network architecture for processing sequences such as text. It works by repeatedly taking a sequence of token representations, letting each position look at other positions through attention, then transforming those representations again.

The important idea is that each token does not get processed in isolation. Each position can build a context-aware representation by looking back across the sequence.

For this assignment, the main consequence is simple: the numbers for one token are allowed to depend on earlier tokens, and attention is the mechanism that decides how much each earlier token matters.

One layer of a Transformer

One Transformer layer is usually described as having two main parts:

1. an attention block
2. a feed-forward block

wrapped with residual connections and normalisation .

INPUT SEQUENCE

The cat sat

Token + position embeddings

Turn each token into an embedding and preserve its position in the sequence.

During generation

Earlier keys and values are kept in the KV cache .

REPEATED TRANSFORMER BLOCKS

Your assignment

Masked self-attention

Q, K, and V, scores, **mask** , **softmax** , weighted values.

Add + norm

Residual connection plus **normalisation** .

Feed-forward block

A small neural-network block applied separately to each position after attention.

Add + norm

The same residual-and-normalisation pattern happens again.

PREDICTION

Vocabulary scoring

Produce logits for all possible next tokens.

Next token

Pick or sample one output token, then repeat.

The full model repeats Transformer blocks many times. This assignment zooms in on the masked self-attention part inside one block, then reuses cached keys and values during generation.

> **Want To Read Paper-Style Block Diagrams?**
>
> If you want help reading the style of architecture figure that often appears in papers, read Reading LLM architecture diagrams.

Inside the attention block, each token representation is turned into query, key, and value vectors. The model then:

1. compares queries against keys to get scores
2. applies masking
3. turns the scores into weights
4. uses those weights to blend the value vectors into new outputs

That is the exact slice of the architecture that your code implements.

Why attention matters

Attention matters because language depends on context. The right interpretation of a word or token often depends on what came earlier.

For example, in a sentence like:

> The animal did not cross the road because it was tired.

the model needs a way to connect “it” back to earlier context. Attention gives each position a structured way to decide which earlier tokens are most relevant.

Without attention, the model would have a much harder time making those long-range connections. Transformers became important because attention made it easier to model those dependencies in parallel and at scale.

Where this assignment fits

This assignment isolates the attention part of one simplified Transformer block.

You are given:

- token embeddings
- projection matrices `Wq`, `Wk`, and `Wv`
- the prompt mask
- generated token embeddings for Stage 6

You implement:

1. projection into $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$
2. prompt attention scores
3. prompt attention weights
4. prompt attention outputs
5. generated outputs using the KV cache

You do **not** implement:

- tokenisation
- training
- multiple attention heads

- feed-forward networks
- residual connections
- layer normalisation
- vocabulary scoring and next-token selection

That is why the assignment feels smaller than a “full Transformer” while still being a real part of one.

> **One-line summary**
>
> The shortest accurate summary is: this assignment asks you to implement the part of a Transformer that decides which earlier positions matter, then mixes information from those positions into a new output vector.

Generation and the KV cache

When a Transformer generates text, it does not want to recompute all previous keys and values from scratch at every step. Instead, it stores them in a KV cache.

That is what Stage 6 is modelling.

At generation step `t`, the model:

1. computes the new token’s query, key, and value
2. appends the new key and value to the cache
3. scores the new query against the cached keys
4. forms the new output from the cached values

This is still attention. The only difference is that the earlier keys and values are reused instead of recomputed.

If you want the assignment-specific version of that story, continue to the KV cache explainer.

COMP10002 FOUNDATIONS OF ALGORITHMS

Reading LLM Architecture Diagrams

How to read the block diagrams that often appear in Transformer and LLM papers, without getting lost in all the boxes and arrows.

Short answer

When you see an LLM architecture figure in a paper, do not try to understand every box at once.

Read it as a data-flow diagram:

1. what goes in
2. what gets repeated
3. what the main sub-blocks are
4. what comes out

Most architecture figures are trying to show the *shape of the computation*, not the full implementation detail.

How to read them

Start with the arrows.

They usually tell you the highest-level story:

- tokens go in
- tokens become embeddings
- those embeddings pass through repeated Transformer blocks
- the final representation is used to produce scores over the next token

Then identify what kind of diagram it is.

Most paper figures are doing one of these jobs:

- showing the overall pipeline from input text to output text
- zooming into one Transformer block
- zooming further into one attention sub-block
- comparing two architectures side by side

If you know which of those jobs the figure is doing, it becomes much easier to read.

Hover or focus the main boxes in the figure below for short plain-English explanations of the main parts.

Transformer full architecture diagram with an encoder stack on the left and a decoder stack on the right, including self-attention, cross-attention, feed-forward blocks, norms, and embedding or projection stages.

A typical Transformer-style paper figure compresses a large computation into a few labelled boxes and repeated stacks. Read it first as a pipeline, then zoom into the sub-block you care about. Figure from **"Transformer, full architecture"** by dvgodoy, licensed **CC BY 4.0**, via Wikimedia Commons.

Look for the input and output first

Find the leftmost or bottommost input, then trace where the arrows eventually lead. That gives you the big picture before you worry about details.

Notice what is repeated

If a diagram says something like "×12", "×24", or "N layers", it usually means "this same block is stacked many times".

Read box labels as roles

Labels such as "self-attention", "feed-forward", "add & norm", or "MLP" are usually naming the role of a sub-computation, not giving all the loop details.

Repeated blocks

A common source of confusion is that one box in a paper figure may really stand for a large repeated stack.

For example, if you see:

- one Transformer block with a note like `× 12`
- a bracket saying `L layers`
- a tall repeated stack in the middle of the figure

that usually means the model applies the same *kind* of block many times in sequence.

The figure is not claiming there is literally only one attention computation. It is compressing many similar layers into one readable visual unit.

That is the same reason papers often draw one attention block, one feed-forward block, and one output head even though the real implementation may involve:

- many layers
- multiple attention heads
- batch dimensions
- training-only components not shown in the simplified figure

What diagrams often leave out

Paper diagrams are helpful, but they often omit the details you would need to code the model directly.

What commonly gets suppressed:

- exact array shapes
- whether vectors are rows or columns
- where masking happens
- whether softmax is row-wise
- how many heads exist inside “multi-head attention”
- what is different at training time versus generation time

> **Why Figures Feel Simpler Than Code**
>
> This is why a paper figure can feel easy to look at but hard to implement from directly. The figure tells you the broad structure; the equations and prose carry the operational detail.

If you want the equation-side version of the same problem, read Reading Transformer paper notation.

How this maps to the assignment

The architecture diagrams in papers usually contain much more than this assignment asks you to implement.

In a typical LLM figure, your assignment corresponds to only a small middle slice:

- building `Q`, `K`, and `V`
- computing attention scores
- applying masking
- applying stable softmax
- using the weights to produce an attention output
- reusing earlier keys and values through the KV cache

That is why the Transformer explainer is useful before or after this page: it shows the larger block structure, while this page is about how to *read* the style of figure that papers tend to use.

COMP10002 FOUNDATIONS OF ALGORITHMS

Transformer History

Historical context for why attention and Transformers mattered, separated from the main implementation path so it stays readable rather than overwhelming.

Neural networks and AlexNet

Neural networks are programs built from many layers of simple numeric computations. A common layer looks like:

$$\text{output} = f(W\,\text{input} + b)$$

where:

- W is a weight matrix
- b is a bias vector
- $f(\cdot)$ is an activation function applied elementwise

That should already look a little familiar from this assignment. Your projection stages are also matrix multiplications with learned weights. In the full Transformer story, many of the important pieces are still built from the same basic ingredients:

- matrix multiplies to form `Q`, `K`, and `V`
- dot products to compare positions
- row-wise softmax to turn scores into weights
- weighted sums to produce new representations

So one way to read Transformers historically is: they are not a rejection of neural networks. They are a new neural-network architecture built from the same kind of numeric primitives, but arranged in a way that handles sequence context much better.

What “non-linear” means in plain English

Why bother with a non-linear `f` at all? Because if every layer were only matrix multiplies and additions, many layers would collapse into one big linear transform. The non-linear step stops that collapse and lets the model represent much richer behaviour.

Common modern activations include:

- ReLU: `max(0, x)`
- GELU: a smoother variant used in many Transformers

You do not implement those activations in this assignment, but they are part of the larger context around Transformer models.

In 2012, AlexNet helped convince the field that deep neural networks plus enough data and computation could work extremely well in practice. That wider deep-learning shift is part of the story that made later architectures like Transformers worth scaling up.

AlexNet itself was not a Transformer, and it was built for images rather than text. But historically it mattered because it pushed the field toward a mindset that also underlies modern large language models:

- large learned weight matrices can capture useful structure
- stacking many simple numeric layers can produce surprisingly strong behaviour
- hardware-friendly linear algebra operations can scale well when the data and compute are available

The 2017 paper

The Transformer architecture became widely known after ***Attention Is All You Need*** (Vaswani et al., 2017).

Its practical message was simple but powerful: strong sequence models did not have to process text strictly one token at a time. Attention could compare many positions with many other positions inside a layer, which fit modern matrix-heavy hardware much better.

This is exactly where the assignment connects back to the history:

- your Stage 2 builds the projections `Q`, `K`, and `V`
- your Stage 3 computes the scaled attention scores
- your Stage 4 applies row-wise stable softmax
- your Stage 5 forms the weighted output vectors
- your Stage 6 uses the KV cache that real autoregressive generation also relies on

So the page is not just saying “Transformers mattered”. It is saying that the algorithmic pipeline you are coding is a stripped-down version of the mechanism that made Transformers so important.

If you want to read around that moment directly:

- The Illustrated Transformer gives a strong visual overview of the architecture
- Attention Is All You Need is the original paper itself

Before vs after

Before Transformers, many sequence models updated an internal state step by step as they read text. That works, but:

- information must travel through many sequential steps
- long-range dependencies are harder to handle
- parallel hardware is harder to use efficiently

After Transformers:

- a layer can compare many token positions in parallel
- the architecture maps naturally onto large matrix multiplications
- scaling the model and data often keeps paying off

That second point is why so much of the Transformer pipeline looks like matrix algebra. The big practical win was not only that attention could model long-range context better, but also that its core operations were things hardware was already very good at:

- matrix multiplies for projections
- batched score computations
- row-wise softmax normalisation
- weighted sums for the final outputs

Why the impact took time

The 2017 paper was framed around machine translation, not around a general-purpose chatbot. It took time for the community to realise that the same architecture, when scaled with more data and more compute, could become a much more general language model.

Part of that delayed impact is that the individual ingredients do not look magical on their own. Matrix multiplies, dot products, masking, and softmax are all fairly ordinary mathematical operations. What turned out to matter was the way those pieces were combined inside the attention mechanism and then scaled aggressively.

In hindsight, Transformers were one of those ideas whose full consequences were clearer only after years of scaling, experimentation, and productisation.

COMP10002 FOUNDATIONS OF ALGORITHMS

Parameters and Model Scale

What “billions of parameters” means, where those numbers come from, and what they imply for memory.

What parameters are

When people say an LLM has “7 billion parameters”, they mean the model stores roughly 7 billion learned numbers.

These are not prompt-specific inputs. They are the model’s learned weights after training.

A compact mental model of parameters

An LLM is a big bundle of matrices full of numbers. At runtime you feed vectors in, the model multiplies by those matrices and applies simple functions, and training is the process of finding good values for those stored numbers.

Where parameters are used

In a Transformer-style model, parameters include:

- the embedding table
- the attention projection matrices (`Wq`, `Wk`, `Wv`, and usually an output projection)
- the feed-forward layers in each Transformer block
- the final layer that maps internal vectors to vocabulary scores

Size examples

Toy-sized example

Suppose:

- vocabulary size $V = 10{,}000$
- model width $d = 512$
- layer count $L = 6$
- feed-forward width $d_{\text{ff}} = 2048$

Then rough parameter counts look like:

- embedding table: $10{,}000 \times 512 \approx 5.12$ million
- attention projections per layer: $4(512 \times 512) \approx 1.05$ million
- feed-forward per layer: $(512 \times 2048) + (2048 \times 512) \approx 2.10$ million

That already gets you to roughly 24 million parameters even before counting every detail.

Rough 7B-style example

Suppose:

- $V \approx 32{,}000$
- $d = 4096$
- $L = 32$
- $d_{\text{ff}} \approx 16{,}384$

Then the model quickly becomes huge:

- embeddings: about 131 million
- attention projections across layers: about 2.1 billion
- feed-forward layers across layers: about 4.3 billion

That is how you arrive at a "7B-ish" model size even with rough counting.

Real model examples

These examples are useful mainly for scale intuition. They are not a leaderboard, and parameter count is only one part of what makes a model strong.

GPT-3

OpenAI, 2020. GPT-3 is a good historical anchor because its paper made the scale very visible: 175 billion parameters.
It is older than current frontier models, but it is still one of the clearest public examples of what "hundreds of billions of parameters" looks like in practice.

Llama 3.1 405B

Meta, 2024. This is a modern open-weight example: 405 billion parameters, with a 128K context window .
It is useful here because it shows that open models now also exist at very large scales, not just small classroom or hobby sizes.

DeepSeek-V3

DeepSeek, 2024. This is a useful modern public example because its technical report gives concrete scale numbers: 671 billion total parameters, with about 37 billion activated per token.

It is also a good reminder that “model size” is no longer always one simple dense parameter count. Modern large models are often sparse or mixture-of-experts systems, so total parameters and active parameters can differ a lot.

So when people talk about model scale today, they may be referring to any mix of:

- parameter count
- context window size
- whether the model is open-weight or closed-source
- whether the architecture is dense or sparse

Memory estimates

Memory for stored parameters is roughly:

$$\text{parameter count} \times \text{bytes per number}$$

Common storage choices:

- FP16 or BF16: about 2 bytes per parameter
- FP32: about 4 bytes per parameter

So a 6.5B-parameter model is roughly:

- about 13 GB in 16-bit storage
- about 26 GB in 32-bit storage

What those memory numbers leave out

Two caveats matter:

- that estimate only counts stored weights, not activations or caches
- training usually needs much more memory than inference because optimisation algorithms keep extra arrays around

For this assignment, the scale is tiny by comparison: `d <= 64`, the matrices are given to you as input, and you do not train anything.

COMP10002 FOUNDATIONS OF ALGORITHMS

How This Assignment Was Built

A short note on why this assignment was developed as a human-AI collaboration, and why that improved the final result for students from Dr. Cohney's perspective.

Short answer

This assignment was built as a human-AI collaboration between Dr. Cohney, Dr. Cohney's AI tool and Kacie Beckett.

The teaching goals, the algorithmic structure, the correctness requirements, the marking priorities, and the final editorial decisions are human. AI was used as a fast design and production partner during drafting, iteration, rewriting, visual explanation, and polish.

> **Why Do This At All?**
>
> The blunt version is: using AI let me build a **fancier, cooler, more interesting, more realistic, and more pedagogically valuable** assignment than I could have produced alone under the same time and energy constraints.

Why use AI here

The point was not to remove human thinking. The point was to increase the amount of thoughtful work that could actually fit into one assignment.

With AI helping, it became practical to build:

- a web-first spec rather than a single flat handout
- linked concept explainers for ideas such as softmax, masking, and attention
- alternate explanations aimed at different levels of prior knowledge
- more diagrams, examples, glossary support, and implementation guidance
- tighter iteration on wording, sequencing, structure, and visual design

That matters because a lot of assignment quality is not in the core algorithm alone. It is in the surrounding teaching infrastructure: how clearly the task is introduced, how well the ideas connect, how easy it is to recover when confused, and how much support exists.

Division of labour

Human responsibilities

The overall concept, learning objectives, course fit, stage decomposition, grading logic, and the final decisions about what belongs in the assignment all remained human responsibilities.

AI responsibilities

AI helped generate alternatives, rewrite explanations, draft examples, propose page structures, help with visuals, and speed up many rounds of revision and cleanup.

What that changed

Instead of stopping at a basic workable spec, the assignment could become a richer teaching artifact with optional background pages, polished concept links, and more thoughtful scaffolding around the core code task.

More concretely, AI was useful for work such as:

- producing multiple candidate phrasings for hard ideas
- helping turn dense technical ideas into first-year-friendly explanations
- suggesting diagrams and visual structures for pages
- speeding up page editing, consistency checks, and refinement across the site
- making it feasible to keep improving the assignment after the minimum viable version already existed

At the same time, important things were deliberately not handed off:

- deciding what students should and should not implement
- deciding what is pedagogically appropriate for a first-year algorithms subject
- deciding what counts as good code structure and good reasoning
- deciding when an explanation is actually clear rather than merely fluent
- deciding what tradeoffs matter between realism, complexity, workload, and fairness

Why students benefit

The result is not just “an assignment with AI somewhere in the workflow”. The point is that the assignment can be better in ways students actually feel.

It can be more interesting because it ties a real modern idea to a manageable programming task. It can be more realistic because collaborating with AI is now part of real technical work. It can be more pedagogically valuable because the surrounding support material can be deeper, more varied, and more responsive to likely confusion points.

It also creates room for something that is often hard to achieve at once:

- a task that still expects genuine student reasoning
- a spec with stronger conceptual support
- a presentation that feels more like a serious technical artifact than a rushed worksheet

That combination is the real reason for the collaboration. The goal was not novelty for its own sake. The goal was to make a better assignment.

> **One Additional Note**
>
> One additional note: Kacie thinks the skills loss that can come with AI use is real, and she should not be blamed for any AI-related mishaps attached to this assignment. Those can be blamed on Dr. Cohney.